VeSAFL: Verifiable Secure Aggregation for Privacy-Preserving Federated Learning

Jaouhara Bouamama, Yahya Benkaouz, and Mohammed Ouzzif

***Abstract*—With the proliferation of IoT devices and the exponential growth of data generated at the edge, federated learning (FL) emerges as a powerful method for training machine learning models on decentralized data sources. In security-critical applications, such as anomaly and threat detection, ensuring the confidentiality and integrity of sensitive data is paramount. In this article, we introduce VeSAFL, a novel scheme for verifiable secure aggregation for privacy-preserving FL designed for edge computing environments. VeSAFL decentralizes model updates across edge nodes, reducing dependency on centralized cloud servers and mitigating risks associated with single points of failure. By leveraging multi-server aggregators, our approach fortifies system resilience and reliability against potential cyber threats. To bolster trust in the learning process, we implement a robust verification mechanism that guarantees the integrity and authenticity of local and global updates. Our experimental results highlight the efficacy and efficiency of VeSAFL in safeguarding against active adversaries and accurately identifying anomalous activities. Furthermore, our comprehensive security analysis affirms the scheme's correctness, verifiability, and privacy preservation in adversarial scenarios.**

***Index Terms*—Federated learning, privacy, secure aggregation.**

I. Introduction

In THE realm of machine learning, federated learning (FL) has emerged as a pivotal paradigm, providing a decentralized method for training models across distributed devices under the coordination of a central server while preserving data privacy [1], [3]. This decentralized approach is particularly significant in today's data-driven era, where the private data of each participant remains stored locally, and only local model updates are communicated, ensuring the non-exposure of sensitive information. The potential of FL to enhance privacy and data security has attracted substantial interest from both academia and industry [5]. FL's capability to leverage data from multiple sources without actual data exchange addresses critical privacy and data sovereignty concerns, making it an ideal solution for sectors like healthcare, finance, and IoT, where data sensitivity is paramount [16], [17], [18], [19].

Received 13 September 2024; revised 8 June 2025; accepted 6 July 2025. Date of publication 14 July 2025; date of current version 4 November 2025. *(Corresponding author: Yahya Benkaouz.)*

Jaouhara Bouamama and Mohammed Ouzzif are with the Computer Science & Smart Systems Laboratory, Higher School of Technology, Hassan II University of Casablanca, Casablanca, Morocco (e-mail: jaouhara.bouamama-etu@etu.univh2c.ma; ouzzif@gmail.com).

Yahya Benkaouz is with the Conception & Systems Laboratory, Faculty of Sciences, Mohammed V University in Rabat, Rabat, Morocco, and also with the College of Computing, Mohammed VI Polytechnic University, Benguerir, Morocco (e-mail: yahya.benkaouz@fsr.um5.ac.ma, yahya.benkaouz@um6p.ma).

Digital Object Identifier 10.1109/TDSC.2025.3589160

As edge computing continues to gain traction, its synergy with FL becomes increasingly apparent. Both paradigms emphasize decentralized architectures, reducing dependency on centralized data processing and enhancing scalability. By processing data closer to the source, edge computing reduces latency and bandwidth usage, offering substantial benefits for security-sensitive applications such as anomaly and threat detection. This decentralized processing is critical in mitigating risks associated with data breaches and cyberattacks.

Despite these advancements, FL in edge computing environments faces significant challenges, particularly regarding data privacy and security. Ensuring privacy in FL within resource-constrained edge environments requires robust and efficient privacy-preserving techniques. Traditional methods, such as Secure Aggregation (SA) [9], aim to aggregate model updates while maintaining the privacy of individual contributions. Techniques like homomorphic encryption [13] and differential privacy [14] have been employed to safeguard data privacy during computation. Homomorphic encryption allows computations on encrypted data, preserving privacy, while differential privacy adds controlled noise to data or model updates, preventing the reconstruction of individual data points from aggregated results. These techniques provide essential privacy guarantees but often face scalability and efficiency issues, particularly in resource-constrained environments.

To address these challenges, we propose VeSAFL, a novel scheme that leverages verifiable secure aggregation for privacy-preserving FL in edge computing. VeSAFL employs a multi-aggregator architecture to decentralize the aggregation process, avoiding single points of failure and enhancing system robustness. This architecture distributes the computational load across multiple aggregators, ensuring efficient resource utilization and reducing the risk of bottlenecks. Furthermore, we introduce a verifiable authenticated sharing protocol, which combines a verifiable secret sharing scheme and short signatures to ensure the privacy and authenticity of local and aggregated updates. The verifiable authenticated sharing protocol enhances the security of the aggregation process by enabling servers to verify the trustworthiness of the node's local updates. Also, nodes can verify the correctness and authenticity of aggregated updates without compromising the privacy of their contributions [28]. This scheme ensures data integrity and fosters trust among nodes and servers, which is essential for the widespread adoption of FL in real-world applications. In summary, our main contributions are as follows:

1545-5971 © 2025 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: Anhui University. Downloaded on October 13,2026 at 08:43:08 UTC from IEEE Xplore. Restrictions apply.

- We design and formalize VeSAFL, a verifiable and privacy-preserving secure aggregation protocol for federated learning, which combines Pedersen's verifiable secret sharing and BLS signatures within a multi-server architecture.
- We provide formal security proofs demonstrating that VeSAFL satisfies correctness, privacy, and robustness in the presence of malicious clients and servers under a defined threat model.
- We implement VeSAFL and evaluate it on MNIST, CIFAR-10, and FEMNIST under IID and non-IID settings. The results show that our scheme achieves competitive model accuracy with strong verifiability guarantees and acceptable computational and communication overhead.

The rest of the paper is organized as follows: Section II reviews the related works. In Section III, we introduce the basic preliminaries used in VeSAFL protocols. Section IV details the system and threat models. The detailed design of the proposed VeSAFL scheme is presented in Section V. Sections VI and VII are dedicated respectively to the security analysis and the experimental evaluation. Section VIII presents discussions and future directions. Finally, Section IX concludes our paper.

II. Related Works

Recently, the convergence of FL with Edge Computing has attracted a lot of interest in providing effective machine learning models. This convergence addresses a growing demand for distributed intelligence while mitigating the privacy impact of traditional centralized cloud computing. In this work, we integrate FL techniques into edge computing environments, focusing on protecting privacy and ensuring the verifiability of model updates. In the following, we provide a concise literature review, focusing on two main directions related to ensuring privacy and verifiability in FL within edge computing environments.

A. Privacy-Preserving Federated Learning in Edge Computing

FL in edge computing environments focuses on optimizing model training while addressing resource constraints and data privacy. Kairouz et al. [16] provide a comprehensive overview of FL advancements, highlighting communication-efficient techniques and privacy-preserving mechanisms. They emphasize the importance of adapting FL algorithms to resource-limited edge devices, proposing model compression and sparse communication solutions. Several techniques have been developed to enhance privacy, ensure communication efficiency, and protect against adversarial threats in FL systems deployed on edge networks. In [2], the authors introduce federated optimization as a distributed machine learning approach suitable for on-device intelligence, emphasizing communication efficiency and model aggregation. In the same direction, McMahan et al. [3] investigate communication-efficient learning of deep networks from decentralized data, introducing methods such as federated averaging and quantization to reduce communication overhead. Additionally, Fotohi et al. introduced a lightweight and secure federated deep learning model tailored for resource-constrained environments, addressing privacy concerns against poisoning and inference attacks while solving the data type distribution skew [4]. In addition, a notable study by Yang et al. [5] explores the use of FL in mobile edge networks, emphasizing privacy preservation through encryption and decentralized learning. The authors highlight the trade-offs between privacy and utility and propose adaptive mechanisms to balance these concerns effectively. To address privacy concerns, Secure Multi-Party Computation (SMPC) offers a layer of privacy protection by enabling multiple parties to jointly compute a function over their inputs while keeping those inputs private. Bonawitz et al. [6] introduce secure aggregation protocols tailored for FL, enabling participants to collaboratively train models without revealing individual updates. Their work demonstrates the potential for secure aggregation to protect data privacy, even in adversarial settings. In the same direction, EdgeSA [7] leverages secure aggregation for privacy-preserving FL in edge computing, combining homomorphic encryption and secure multi-party computation to provide robust privacy guarantees. In [8], a new system called SecureML for scalable privacy-preserving machine learning is introduced. SecureML employs SMPC protocols to enable secure collaborative training. The scheme is based on additive sharing, boolean sharing and Yao sharing to allow secure computation of machine learning models without revealing any participant's data. On the other hand, Truex et al. [10] propose a hybrid FL approach that combines differential privacy and secure multi-party computation. This approach aims to ensure robust privacy guarantees while maintaining model accuracy. Furthermore, McMahan et al. [11] introduced the concept of differentially private FL by adding controlled noise to model updates, thus providing privacy guarantees without significantly impacting model performance. Their work studied the effectiveness of integrating differential privacy into FL models to mitigate privacy risks. Wei et al. [12] extended this approach by developing a privacy-preserving FL framework specifically for IoT devices, leveraging local differential privacy to maintain privacy at the edge. For more related works about privacy-preserving FL readers may refer to [15], [45].

B. Verifiable Federated Learning

Ensuring the integrity and trustworthiness of model updates in FL is essential for maintaining the reliability of collaborative learning processes. Verifiable Federated Learning (VFL) has emerged as a crucial approach to address these concerns by incorporating mechanisms that verify the correctness and authenticity of updates without compromising privacy [37]. One of the first frameworks introduced in this area was VerifyNet [29], which uses a double-masking mechanism to ensure the confidentiality of local model updates. In this scheme, the cloud server must provide proof of correctness of the global aggregated result for users. Based on this framework, Han et al. [30] introduce a framework for verifiable and privacy-preserving FL without relying on fully trusted centers. Here, the trusted center is removed and a key agreement algorithm is leveraged instead. The authors also make use of the double masking protocol to ensure the privacy model updates. Additionally, the tag aggregation method is used to ensure the correctness of the results returned

Authorized licensed use limited to: Anhui University. Downloaded on October 13,2026 at 08:43:08 UTC from IEEE Xplore. Restrictions apply.

by the server. In another direction, Fu et al. [31] introduce a verifiable FL scheme that applies Lagrange interpolation to verify aggregated gradients in industrial IoT applications. This method sets interpolation points while applying the Chinese Remainder Theorem to verify the correctness of the aggregated gradients and to reduce communication overhead. Additionally, VERSA [32] is a verifiable secure aggregation framework for cross-device FL, enhancing security by verifying the aggregation of model updates and ensuring that malicious activities are detected. However, in [38], the authors revealed the flaw of VERSA through mathematical analysis. Moreover, Zhang et al. [33] propose VPFL, a verifiable privacy-preserving FL scheme for edge computing systems. This scheme prevents gradients from leaking over the transmission phase by using distributed Selective Stochastic Gradient Descent (DSSGD) method with Paillier homomorphic cryptosystem to achieve the distributed encryption functionality. VPFL is enhanced by online/offline signature method to realize the lightweight gradients integrity verification. Similarly, schemes supporting verifiability of aggregated results include VOSA, a verifiable and oblivious secure aggregation protocol for privacy-preserving FL [34]. Here, the authors build the scheme over aggregator oblivious encryption scheme to mask users' local gradients. Thus, the aggregator aggregates the obscured gradients without revealing the local data. And, each user can verify the correctness of the aggregated results. However, Wu et al. [39] show that VOSA is insecure, in which local gradients/aggregation results and their corresponding authentication tags/proofs can be tampered with without being detected by the verifiers. Additionally, Wang et al. [35] propose a verifiable and privacy-preserving federated learning scheme that combines symmetric homomorphic encryption with a homomorphic signature mechanism based on BLS signatures and asymmetric programmable hash functions. Their approach ensures that local model updates remain confidential during transmission and aggregation, and introduces an efficient verification scheme that allows clients to validate the correctness of the aggregated result without incurring significant overhead. Contrary to the previous schemes where verification is conducted once, [36] which incorporates successive verification phases to enhance security and operational transparency. The verification is conducted based on data auditing where only a portion of data is used to generate proofs. However, the scheme assumes the existence of a trusted authority responsible for data integrity verification and can have additional computational overhead. The previously mentioned schemes assume the existence of a single central server responsible for the aggregation phase. However, this assumption poses the risk of a single point of failure, thereby highlighting the importance of distributed federated learning. Distributed FL can be categorized as either peer-to-peer (P2P) based architectures or multi-server-based architectures. In P2P-based architectures, the blockchain paradigm has emerged as a powerful tool. Specifically, Fotohi et al. [40] proposed a consortium blockchain-enabled FL framework to enhance privacy-preserving FL, emphasizing robustness against poisoning and inference attacks and decentralization without relying on a single aggregator. P2P-based architectures are out of scope in this paper, thus readers can refer to the following surveys [41], [42] for further details about P2P-based architectures in FL. On the other hand, multi-server based architectures distribute the aggregation and verification processes across several servers to enhance robustness and reliability. Xu et al. [43] propose a non-interactive verifiable privacy-preserving FL scheme, based on dual-servers architecture. The authors propose a privacy gradient aggregation scheme by exploiting secure two-party computation to encrypt local gradients. Here, the verification is achieved by random matrix coding, where the dual-servers realize cross-verification and credible matrix exchange for the verification of the aggregated result. However, the scheme assumes the non-collusion between the dual-servers. Similarly, Brunetta et al. [44] introduce NIVA, a non-interactive, secure verifiable aggregation framework for decentralized, privacy-preserving learning. This scheme ensures distributed aggregation of multiple users secret inputs by multiple untrusted servers. The returned aggregation result can be publicly verified in a non-interactive way. However, these works generally assume a weaker threat model, where users and servers are assumed to be semi-honest and not colluding, and only the global aggregated update is subject to verification. Unlike existing frameworks that rely on centralized aggregation, VeSAFL adopts a multi-aggregator architecture for enhanced robustness. By integrating advanced cryptographic techniques such as verifiable secret sharing and bilinear short signatures, VeSAFL ensures efficient verification where both local and global updates are verified. Also, VeSAFL ensures robust privacy guarantees and optimized resource utilization in the edge in a stronger threat model where both servers and nodes are considered untrustworthy.

TABLE I
RELATED WORKS SUMMARY

References	Privacy	Verifiability	Authentication	Robustness	Efficiency
[2]–[12]	✓	×	×	×	✓
[30]–[37]	✓	✓	×	×	✓
[44], [45]	✓	✓	×	✓	×
VeSAFL	✓	✓	✓	✓	✓

Table I summarizes the key features of the referenced works in verifiable and privacy-preserving FL schemes compared to our VeSAFL scheme. Five main features have been considered in this comparison: 1- Privacy: emphasizes safeguarding sensitive data during collaborative model training without undermining local data security. 2- Verifiability: ensures the correctness and authenticity of local and global updates. 3- Authentication: verifies the identity of participants and ensures that only legitimate data is being used in the model updates. 4- Robustness: addresses the system's ability to withstand failures and malicious behavior. 5- Efficiency: focuses on optimizing communication and computation overhead, thus considering edge resource constraints.

III. PRELIMINARIES

In this section, we present the basic background required to introduce our suggested scheme. This includes: Federated learning formalization, verifiable secret sharing schemes, and bilinear short signature.

Authorized licensed use limited to: Anhui University. Downloaded on October 13,2026 at 08:43:08 UTC from IEEE Xplore. Restrictions apply.

A. Federated Learning

Federated learning is a decentralized machine learning approach that allows models to be trained on several devices while maintaining data privacy. Unlike typical centralized machine learning, which collects and stores data on a single server for training, FL allows training to occur on the devices themselves. Thus, the model is trained on each local device utilizing locally stored data, without sharing the raw data with the central server. Only model updates, often in the form of weights, are sent to the central server. The central server aggregates those model updates and sends the global aggregated model back to each local device.

Formally, let us assume a set of n participants $i = 1, \ldots, n$ each holding a local dataset $D_i = \{(x_i, y_i) | i = 1, \ldots, |D_i|\}$, such that x_i is the input and y_i is the label. Let $f(M, x) = \hat{y}$ be the neural network model with a model M, an input x, and a predicted label $\hat{y}$. Let $L_f(y, \hat{y})$ be the loss function that measures the difference between the true label y and the predicted label $\hat{y}$. The objective of training a neural network is to determine the parameters (e.g., weights) ω to update the model M to minimize the loss function $L_f(D_i, M)$:

$$L_f(D_i, M) = \frac{1}{|D_i|} \sum_{(x_i, y_i) \in D_i} \left(y_i - f\left(M, x_i\right)\right)^2.$$

As a starting point, the central server initializes the global model weights ω_0 and sends them to all participating devices. At each round, each participant i updates its model weights ω_i by performing a stochastic gradient descent using his local dataset D_i:

$$\omega_i = \nabla L_f(D_i, M),$$

where, ∇L_f is the derivative of the loss function L_f.

After each round of local updates, the central server aggregates the model updates received from all participants :

$$\omega = \sum_{i=1}^{n} \omega_i.$$

The central server sends the updated global weights ω to all participants, where they can update the model:

$$M = M - \eta \frac{\omega}{n},$$

where η is the learning rate. This process continues until model convergence or a termination condition is met.

B. Verifiable Secret Sharing Schemes

In our proposed scheme, we make use of Pedersen's secret sharing scheme [23] which falls into the category of Verifiable Secret Sharing schemes (VSS). The scheme is considered one of the most used VSS schemes since it is information-theoretically secure (i.e., the information the adversary learns from the shares is independent of the secret). A secret is distributed into n shares and can be reconstructed when a sufficient number (i.e., a threshold k) of shares are combined. Unlike traditional secret sharing scheme properties [20], [21], [22], each participant (shareholder) can verify that his share is a valid share of the original secret. In a nutshell, the scheme aims to share a secret s while making it possible for all participants to verify the correctness of the shares. Using a *commitment scheme*, the secret holder will commit to the secret s using a randomly chosen value r (blinding factor). The secret s is then divided into multiple shares following Shamir's scheme [20]. Before distributing the shares, the secret holder will commit to the shares. Therefore, any participant can verify the consistency of the communicated shares. The verifiability property is defined as follows:

Definition 1: A scheme that involves a secret holder and n participants is verifiable if it satisfies the following requirements:

1) If all parties (secret holder and participants) are honest and follow the sharing and verification protocols, then each participant P_i accepts his share with probability 1.
2) For random participants subsets S_1 and S_2 of size k each, such that participants $(P_i)_{i \in S_1}$ and $(P_i)_{i \in S_2}$ have accepted their shares. If s_j is the secret to be reconstructed by the participants in S_j (for $j = 1, 2$) then $s_1 = s_2$ except with negligible probability.

Pedersen's verifiable secret sharing scheme [23] involves the following algorithms: **Commit**, **Share**, **Verify** and **Recon**. Considering n servers, a threshold k, a prime number p and a finite field $GF(p)$ with $g, h \in GF(p)$, these algorithms are described as follows:

1) $\mathbf{Commit}(s, r)$: commit to a secret s using a random blinding factor r by computing $g^s h^r$.
2) $\mathbf{Share}(s, r)$: generate shares (s_α, r_α) of s and r respectively, by randomly choosing coefficients a_i and b_i (i.e., b_i is used to commit to a_i) for $i = \{1, \ldots, k-1\}$. Then, create polynomials f and g of degree $k-1$ such that $a_0 = s$ and $b_0 = r$:

$$f(x) = a_0 + a_1 x + \cdots + a_{k-1} x^{k-1} mod q$$

$$g(x) = b_0 + b_1 x + \cdots + b_{k-1} x^{k-1} mod q$$

The shares are $s_\alpha = f(\alpha), r_\alpha = g(\alpha)$ for $\alpha = 1, \ldots, n$. The commitments are of the form:

$$E_i = \mathbf{Commit}(a_i, b_i) \;\text{ for }\; i = \{0, \ldots, k-1\}.$$

Then, the shares (s_α, r_α) for $\alpha = 1, \ldots, n$ are sent privately to the servers while proofs E_i are made public.

3) $\mathbf{Verify}(s_\alpha, r_\alpha)$: Verify the shares by computing :

$$\mathbf{Commit}(s_\alpha, r_\alpha) = E'_\alpha$$

and using public commitments E_i, we evaluate:

$$\prod_{i=0}^{k-1} E_i^{\alpha^i} == E'_\alpha$$

4) $\mathbf{Recon}(\{(s_\alpha, r_\alpha)\}_{\alpha \in \{1, \ldots, k+1\}})$: After verifying the shares, the secret can be reconstructed using at least k valid shares by applying Lagrange polynomial interpolation:

$$f(0) = s = \sum_{\alpha=1}^{k+1} s_\alpha \lambda_\alpha,$$

Authorized licensed use limited to: Anhui University. Downloaded on October 13,2026 at 08:43:08 UTC from IEEE Xplore. Restrictions apply.

$$\lambda_\alpha = \prod_{\substack{1 \le m \le k+1 \\ m \neq \alpha}} \frac{m}{m-\alpha}$$

Note that the following theorems apply to Pedersen's secret sharing scheme and have been proven in [23]:

Theorem 1: Under the assumption that a server cannot break the discrete logarithm problem except with negligible probability, the verification algorithm satisfies Definition 1.

Theorem 2: Given any subset of servers S of size at most $k-1$ and any $view_S$ (the information being held by servers in S):

$$view_S = [E_0, E_1, \ldots, E_{k-1}, (s_i, r_i)_{i \in S}],$$
$$Pr[S \text{ has secret } s \mid views_S] = Pr[S \text{ has secret } s].$$

The choice of Pedersen's VSS scheme in our construction is motivated by several key properties that make it particularly suitable for FL settings [23], [51]:

- *Efficiency:* Pedersen VSS incurs low computational and communication overhead, making it suitable for edge and resource-constrained environments.
- *Non-interactive Verifiability:* The scheme supports verification of share consistency using public commitments, without requiring interaction between the secret dealer and the verifiers.
- *Information-Theoretic Privacy:* Even an adversary with unbounded computational power learns nothing about the secret unless the reconstruction threshold is met, ensuring strong confidentiality guarantees.
- *Robustness:* The scheme remains secure in the presence of malicious or colluding parties. As long as the number of honest participants exceeds the threshold, the secret can be correctly reconstructed.
- *Linearity:* Pedersen shares support homomorphic properties that enable linear operations on shared secrets, a valuable property for secure aggregation protocols.

C. Bilinear Short Signature

Bilinear pairing-based short signatures are cryptographic primitives that provide a compact representation of digital signatures while leveraging the properties of bilinear pairings. Boneh et al. [24] introduced the first bilinear short signature (i.e., BLS short signature) based on the mathematical properties of bilinear pairing. A *bilinear pairing scheme* [25] satisfies the following definition.

Definition 2: Let $\mathbb{G}, \mathbb{G}_T$ be two cyclic groups of prime order p. A bilinear pairing can be represented as a bilinear map $e : \mathbb{G} \times \mathbb{G} \to \mathbb{G}_T$ which satisfies the following properties:

1. *Bilinearity:* given $(g, h) \in \mathbb{G} \times \mathbb{G}$ and $(a, b) \in \mathbb{Z}_p^*$
$$e\left(g^a, h^b\right) = e(g, h)^{ab}.$$
2. *Computability:* $e(g, h)$ can be computed efficiently for any $g, h \; in \; \mathbb{G}$.
3. *Non-degeneracy:* if g generates $\mathbb{G}$ then $e(g, g) \neq 1$.

A group $\mathbb{G}$ is a bilinear group, if the group operation in $\mathbb{G}$ is efficiently computed where there exists a group $\mathbb{G}_T$ along with an efficient-computable map $e : \mathbb{G} \times \mathbb{G} \to \mathbb{G}_T$ as described above. Let us assume we have a bilinear map $e : \mathbb{G} \times \mathbb{G} \to \mathbb{G}_T$, all the cyclic groups are of prime order p and g is a generator of $\mathbb{G}$. The security of the bilinear pairing scheme relies on the q-Strong Diffie-Hellman Problem [25].

a) q-Strong Diffie-Hellman Problem: The q-SDH in $\mathbb{G}$ is defined as follows: given a $(q+2)$-tuple $(g, g^\gamma, g^{\gamma^2}, \ldots, g^{\gamma^q})$ as input, the aim is to compute a pair $(g^{\frac{1}{\gamma+x}}, x)$ where $x \in \mathbb{Z}_p^*$.

An algorithm $\mathcal{A}$ has advantage ϵ in solving q-SDH in $\mathbb{G}$ if

$$Pr\left[\mathcal{A}\left(g, g^\gamma, g^{\gamma^2}, \ldots, g^{\gamma^q}\right) = \left(g^{\frac{1}{\gamma+x}}, x\right)\right] \le \epsilon,$$

where the probability is over the random choice of the generator g in $\mathbb{G}$, of γ in $\mathbb{Z}_p^*$ and of random bits of $\mathcal{A}$.

Given the defined bilinear pairing e and assuming the existence of a full-domain hash function $H : \{0,1\}^* \to \mathbb{G}$, this function is to be treated as a random oracle. A short signature is composed of three algorithms:

1) **Gen**(): generates a public key pk and a private key sk by choosing a random $sk \leftarrow \mathbb{Z}_p^*$ and setting $pk = g^{sk}$.
2) **Sign**(sk, m): given a message $m \in \{0,1\}^*$, computes $h \leftarrow H(m)$ and outputs a signature $\sigma \leftarrow h^{sk}$.
3) **Verify**(pk, m, σ): given a signature σ, a message m and a public key pk, if $e(g, \sigma) = e(pk, H(m))$, the signature is $valid$.

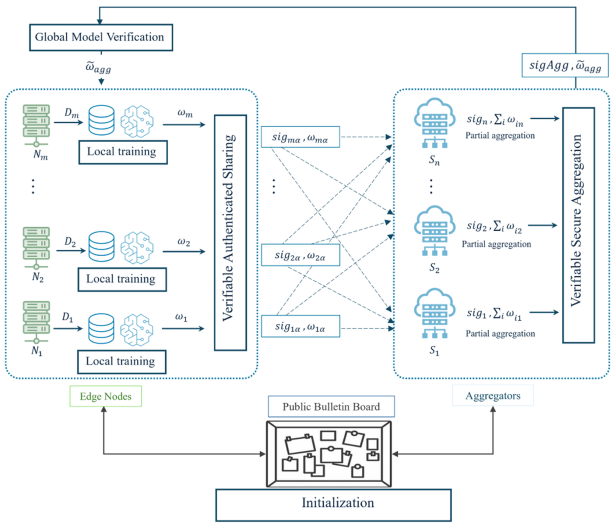


Fig. 1. System model.

IV. System Model and Threat Model

After describing the basic concepts related to our suggested scheme, this section aims to present the considered system and threat models. The parameters used throughout this work are summarized in Table II.

A. System Model

We consider a multi-server architecture to decentralize the aggregation and avoid the single point of failure. Hence, our system model consists of m edge nodes and n servers (a.k.a. aggregators). As shown in Fig. 1, the m edge nodes have the

Authorized licensed use limited to: Anhui University. Downloaded on October 13,2026 at 08:43:08 UTC from IEEE Xplore. Restrictions apply.

TABLE II
VESAFL SCHEME PARAMETERS

Parameters	Definition
	VeSAFL initial parameters
n	Number of servers
k	Threshold
m	Number of edge nodes
d	Number of valid edge nodes
e	Bilinear map
$\mathbb{G}, \mathbb{G}_T$	Cyclic groups
p	Group order
g, h	Group generators
H	Full domain hash function
	Model training parameters
N_i	Node i
α, β	Servers indexes
S_α	Server α
D_i	Node i dataset
ω_i	Node i local parameter
r_i	Node i blinding factor
$\omega_{i\alpha}$	Node i share for sever S_α
$r_{i\alpha}$	Node i blinding factor share for server S_α
$\tilde{\omega}_\alpha$	Server S_α parameter partial aggregation
$\tilde{r}_\alpha$	Server S_α blinding factor partial aggregation
$\tilde{\omega}_{agg}$	Global parameter aggregation
$\tilde{r}_{agg}$	Global blinding factor aggregation
	Cryptographic parameters
λ	Security parameter
pk_α	Server S_α public key
sk_α	Server S_α private key
ϑk_i	Node N_i public key
κ_i	Node N_i private k ey
j	Index in $\{0, ..., k-1\}$
a_{ij}, b_{ij}	Coefficients of secret sharing polynomial
π_{ij}	Proof related to coefficients a_{ij}, b_{ij}
π_i	Node N_i set of proofs π_{ij}
$sig_{i\alpha}$	Node N_i signature of shares
sig_α	Server S_α signature of partial aggregation
$sigAgg_\alpha$	Server S_α signature of global aggregation

responsibility of collecting local data, training the local model, performing the verifiable authenticated sharing scheme to distribute the updates, and publishing proofs in a public untampered bulletin board to maintain verifiable privacy.

The bulletin board is modeled as a public, append-only ledger, ensuring that proofs and public keys, remain immutable and publicly verifiable. We emphasize that this component is an abstraction that can be implemented flexibly according to deployment requirements. For instance, practical implementations may leverage permissionless blockchains such as Bitcoin [47] or Namecoin [48], or permissioned ledger frameworks like Hyperledger Fabric [49], which naturally provide the desired security properties, including integrity, global consistency, and resistance to tampering. Such bulletin boards have been effectively employed in secure cryptographic applications, particularly decentralized Public Key Infrastructures (PKIs) [50].

Upon receiving secret shares from multiple edge nodes, each server runs a verifiable secure aggregation process to verify the correctness of the shares used in the aggregation. Then, instead of reconstructing each local parameter separately, which violates the privacy requirement of local updates, each server computes an authenticated partial aggregation using the valid received shares. Finally, at least a threshold $(k \leq n-2)$ of servers is needed to reconstruct the global aggregated parameter using at least k partial aggregations. Finally, using public proofs, each node can verify the correctness of the global authenticated parameter.

VeSAFL adopts a threshold-based complaint mechanism to mitigate the risk of malicious or faulty participants (i.e., servers and nodes). A participant is excluded only after receiving at least k cryptographically verifiable complaints. Each complaint must be backed by public commitment inconsistencies, enabling local verification by the aggregators. This approach aligns with techniques proposed in classical verifiable secret sharing protocols such as Gennaro et al. [51], where complaints are checked against commitments to prevent arbitrary disqualification.

B. Threat Model

When it comes to FL, there are mainly two threat models that are being considered: 1- *Honest-but-curious* model, which assumes that both the aggregation servers and the nodes follow the protocol honestly, but may try to learn or infer other node's private data. 2- *Active adversary model* assumes that both the aggregation servers and nodes may deviate from the scheme by distributing incorrect shares or sending false aggregation results. In the following, we present the underlying assumptions considered in VeSAFL:

- We assume the existence of a public, append-only, authenticated bulletin board accessible to all participants, ensuring that all posted data (such as public keys, proofs, etc.) cannot be altered or deleted.
- We assume the existence of a complaint system in which any node or server accumulating at least k verified complaints is excluded from the system and no longer receives subsequent global model updates.
- We assume that at most $k-1$ aggregation servers may be malicious (possibly colluding with malicious nodes), where k is the threshold for secret reconstruction.

In VeSAFL, the local parameters are exchanged and shared instead of local data, thus the privacy of the local data is reduced to the privacy of the local parameters. Therefore, the main security requirements being considered in our scheme are:

- *Correctness:* Honest servers and edge nodes can correctly get the global aggregations.
- *Verifiablity:* Prevent malicious servers and edge nodes from sending incorrect or forged information (i.e., local and global updates).
- *Privacy:* Mandates that neither local data nor its associated parameters are disclosed, even in the presence of active adversaries.

V. VESAFL SCHEME

This section aims to describe our suggested VeSAFL scheme. This will go through an overview of the main steps of the scheme, followed by a detailed description of the building blocks, and ending with the protocol implementing each phase.

A. Overview

As shown in Fig. 2, and starting with initialization, we assume the existence of a public tamper-proof bulletin board. The initial

Authorized licensed use limited to: Anhui University. Downloaded on October 13,2026 at 08:43:08 UTC from IEEE Xplore. Restrictions apply.

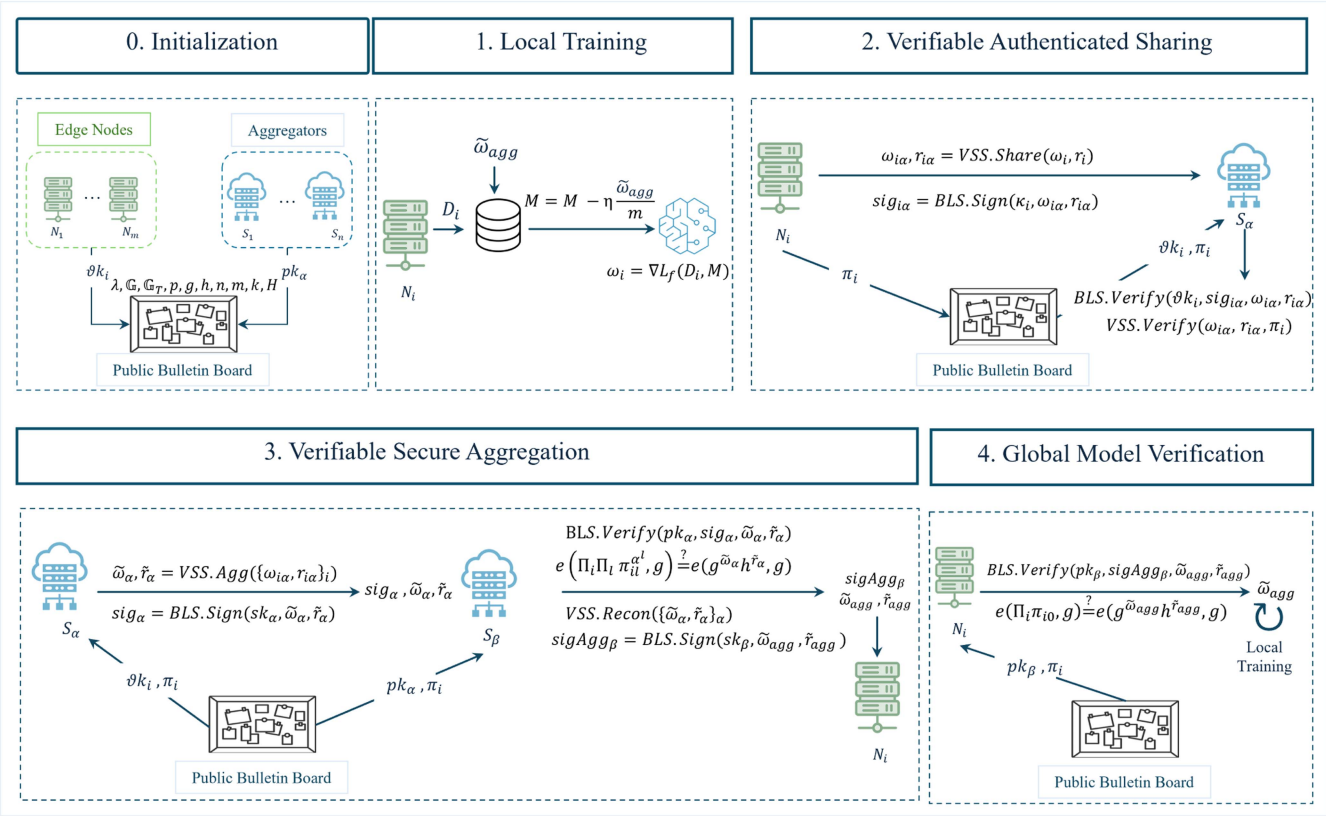


Fig. 2. VeSAFL overview.

public parameters and public keys of both the nodes (N_i, i is the index of the nodes) and the servers (S_α, S_β where α, β are indexes of the servers) are registered in the bulletin board.

The local training block is conducted as in a traditional FL scheme where each node trains a machine learning model M on his local data D_i using a global aggregated parameter $\tilde{\omega}_{agg}$. Then, outputs a new local parameter ω_i.

After obtaining a new local parameter, each node N_i generates a set of verifiable authenticated shares of its parameter using a blinding factor r_i. Each node distributes the signed shares to the different servers. The verification proofs are published on the bulletin board.

Each server S_α starts by verifying the signature of a node N_i making use of the public key, and then verifies the correctness of the shares based on the proofs. If they are successfully verified, the server S_α computes partial aggregations of the received shares and signs it with his private key. Note that any server S_β is able to verify the authenticated partial aggregation. When at least k authenticated partial aggregations are correctly verified, a server S_β can reconstruct the global parameter aggregation. The global parameter aggregation is then sent to all nodes with an associated signature.

Upon receipt of the global aggregation from a server S_α, a node N_i verifies the signature and the correctness of the global aggregation using the public key and the proofs from the bulletin board. The correctly verified global parameters are used again in the local training step until model convergence. Let us recall that the public bulletin board maintains all servers public keys pk_α and nodes public keys ϑk_i, corresponding to their secret keys sk_α and κ_i respectively. Those keys are generated as follows: $pk_\alpha = g^{sk_\alpha}$ and $\vartheta k_i = g^{\kappa_i}$, for $\alpha = \{1, \ldots, n\}$ and $i = \{1, \ldots, m\}$. Additionally, once a node or a server is subject to more than k verified complaints, he is excluded from the protocols.

Note that the generation of the local authenticated parameter shares is a mandatory step in our protocol. In this step, each node N_i holding a local parameter ω_i, runs a Verifiable Authenticated Secret Sharing instance to generate authenticated shares and the corresponding proofs. This instance is composed of a *verifiable secret sharing* and a *bilinear short signature* scheme, which will be detailed in the next subsection.

B. Building Blocks

To ensure an efficient and verifiable authenticated secure aggregation, our suggested VeSAFL scheme makes use of *Pedersen verifiable secret sharing in bilinear groups* and *pairing-based short signature*.

1) Verifiable Secret Sharing Scheme: We construct Pedersen VSS in bilinear groups as follows: Let $\mathbb{G}, \mathbb{G}_T$ be cyclic groups of order p and let g, h be the generators of $\mathbb{G}$ and $e : \mathbb{G} \times \mathbb{G} \to \mathbb{G}_T$ be a bilinear map. A **VSS** instance is composed of the following algorithms:

- $\mathbf{VSS.Share}(\omega_i, r_i)$: takes as input the local parameter ω_i and a random blinding value r_i and outputs a set of n shares $(\omega_{i\alpha}, r_{i\alpha})$.

Authorized licensed use limited to: Anhui University. Downloaded on October 13,2026 at 08:43:08 UTC from IEEE Xplore. Restrictions apply.

Algorithm 1: VSS.Share.

Input: Local parameter ω_i and a random blinding r_i of a node N_i
Output: $\{\omega_{i\alpha}, r_{i\alpha}\}_{\alpha=1,\ldots,n}, \pi_i$
Let $a_{i,0} = \omega_i$ and $b_{i,0} = r_i$
for $j = 1, \ldots, k-1$ **do**
  Randomly choose $a_{i,j}$ and $b_{i,j}$
end for
Create two polynomials f_i and g_i of degree $k-1$:
  $f_i(x) = a_{i0} + a_{i1}x + \cdots + a_{ik-1}x^{k-1} \mod q$
  $g_i(x) = b_{i0} + b_{i1}x + \cdots + b_{ik-1}x^{k-1} \mod q$
for $\alpha = 1, \ldots, n$ **do**
  Let $\omega_{i\alpha} = f_i(\alpha)$ and $r_{i\alpha} = g_i(\alpha)$
end for
Let $\pi_i = \mathbf{VSS.Proof}(\{a_{ij}, b_{ij}\}_{j=0,\ldots,k-1})$
Return: $\{\omega_{i\alpha}, r_{i\alpha}\}_{\alpha=1,\ldots,n}, \pi_i$

- $\mathbf{VSS.Proof}(a_{ij}, b_{ij})$: takes as input the coefficients a_{ij} and b_{ij} used in the sharing process and outputs a set of k proofs of correctness (i.e., $\pi_i = \{\pi_{ij}\}_{j=\{0,\ldots,k\}}$).
- $\mathbf{VSS.Verify}(\omega_{i\alpha}, r_{i\alpha}, \pi_i)$: verifies whether the shares are correctly computed from the committed coefficients in the proofs and outputs a boolean value.
- $\mathbf{VSS.Agg}(\{(\omega_{i\alpha}, r_{i\alpha})\}_{i=\{1,\ldots,d+1\}})$: takes as input d valid shares (i.e., $d \leq m$) and outputs partial aggregation $(\tilde{\omega}_\alpha, \tilde{r}_\alpha)$.
- $\mathbf{VSS.Recon}(\{\tilde{\omega}_\alpha, \tilde{r}_\alpha\}_{\alpha=\{1,\ldots,k+1\}})$: takes as input k valid partial aggregations $(\tilde{\omega}_\alpha, \tilde{r}_\alpha)$ and outputs the reconstructed values $(\tilde{\omega}, \tilde{r})$.

Algorithms 1, 2, 3, 4, and 5 describe the verifiable secret-sharing implementations designed for our VeSAFL scheme. The first algorithm describes the process of sharing the local parameter of a node N_i among α servers and makes use of Algorithm 2. This latter presents a proof function. This function ensures that a node N_i cannot alter the local parameter once the shares have been distributed. The Algorithm 2 produces a proof of correctness in the form of commitments. Algorithm 3 implements the verification step to check the correctness of each share. A valid share should be generated from the committed coefficients that produced the proofs. The computation of partial aggregation of valid shares follows Algorithm 4, where each server computes a partial aggregation using only the valid shares. Finally, Algorithm 5 reconstructs the global parameter. Each set of a least k servers can collaborate to reconstruct the global parameter directly from partial aggregations and thus do not need direct access to local parameters.

2) Bilinear Short Signature: Suppose a participant P has a message m and a public-private key pair (pk, sk), each participant obtains a signature sig of the message m using sk which is computationally costly. In our suggested scheme, both nodes and servers can run a Bilinear short signature instance to produce short signatures. The goal is to achieve efficient verifiability against malicious adversaries. In the following, we briefly describe the construction used in our scheme.

Algorithm 2: VSS.Proof.

Input: Coefficients $\{a_{ij}, b_{ij}\}_{j=0,\ldots,k-1}$ of a node N_i
Output: Proofs π_i
for $j = 0, \ldots, k-1$ **do**
  $\pi_{i,j} = g^{a_{ij}} h^{b_{ij}}$
end for
Let $\pi_i = \{\pi_{i,j}\}_j$
Return: π_i

Algorithm 3: VSS.Verify.

Input: Set of proofs π_i of a node N_i, a share $(\omega_{i\alpha}, r_{i\alpha})$
Output: True if $\omega_{i\alpha}, r_{i\alpha}$ are valid, False otherwise
if $e(\prod_{l=0}^{k-1} \pi_{il}^{\alpha^l}, g) == e(g^{\omega_{i\alpha}} h^{r_{i\alpha}}, g)$ **then**
  Return: True
else
  Return: False
end if

Algorithm 4: VSS.Agg.

Input: Valid shares $\{\omega_{i\alpha}, r_{i\alpha}\}_{i=1,\ldots,d+1}$
Output: Partial aggregation $(\tilde{\omega}_\alpha, \tilde{r}_\alpha)$
Let $\tilde{\omega}_\alpha = \sum_{i=1}^{d+1} \omega_{i\alpha}$
Let $\tilde{r}_\alpha = \sum_{i=1}^{d+1} r_{i\alpha}$
Return: $(\tilde{\omega}_\alpha, \tilde{r}_\alpha)$

Algorithm 5: VSS.Recon.

Input: Valid partial aggregations $\{\tilde{\omega}_\alpha, \tilde{r}_\alpha\}_{\alpha=1,\ldots,k+1}$
Output: Global parameter $\tilde{\omega}_{agg}$, global blinding factor $\tilde{r}_{agg}$
Compute:

$$\tilde{\omega}_{agg} = \sum_{\alpha=1}^{k+1} \tilde{\omega}_\alpha \lambda_\alpha, \quad \tilde{r}_{agg} = \sum_{\alpha=1}^{k+1} \tilde{r}_\alpha \lambda_\alpha$$

$$\lambda_\alpha = \prod_{\substack{1 \leq i \leq k+1 \\ i \neq \alpha}} \frac{i}{i-\alpha}$$

Return: $\tilde{\omega}_{agg}, \tilde{r}_{agg}$

Let us consider $\mathbb{G}$ a cyclic group of prime order p and $e : \mathbb{G} \times \mathbb{G} \to \mathbb{G}_T$ a bilinear map, and $H : \{0,1\}^* \to \mathbb{G}$ a full domain hash function. This function is to be treated as a random oracle [24].

Given a public-private key pair (pk, sk), A **BLS** instance consists of the following algorithms:

- $\mathbf{BLS.Sign}(sk, m)$: takes as input a private key and a message and outputs a signature sig.
- $\mathbf{BLS.Verify}(pk, sig, m')$: takes as input a public key, a signature, and the received message m' and verifies whether the signature is correct or not.

Without loss of generality, a participant P can be a node N_i or a server S_α and a secret key *SK* is either sk_α or κ_i and a public key *PK* is either pk_α or ϑk_i for nodes and servers respectively.

Authorized licensed use limited to: Anhui University. Downloaded on October 13,2026 at 08:43:08 UTC from IEEE Xplore. Restrictions apply.

Algorithm 6: BLS.Sign.

Input: Private key *SK* of a participant and a secret value M
Output: Signature *SIG*
Compute $h = H(M)$
Let $SIG = h^{SK}$
Return: *SIG*

Algorithm 7: BLS.Verify.

Input: Public key *PK*, *SIG*, and secret value M'
Output: True if M' is valid, False otherwise
Compute $h = H(M')$
if $e(PK, h) == e(g, SIG)$ **then**
  Return: True
else
  Return: False
end if

A secret value M is either a share tuple $(\omega_{i\alpha}, r_{i\alpha})$ or a partial aggregation tuple $(\tilde{\omega}_\alpha, \tilde{r}_\alpha)$.

In VeSAFL, we assume that all participants have registered their public keys. Hence, the short signature scheme is composed of Algorithms 6 and 7. Algorithm 6 aims to sign a secret value M. It takes as input a private key *SK* and a secret value M and outputs a related signature *SIG*. Whereas Algorithm 7 verifies the validity of a given signature.

C. Detailed Description of VeSAFL Scheme

Our proposed VeSAFL scheme (depicted in Fig. 3) explicitly consists of five distinct phases. The first phase, Initialization, is executed only once at the beginning of the protocol. This is followed by four main recurring phases: local training, verifiable authenticated sharing, verifiable secure aggregation, and global model verification.

- *Initialization*: The initialization process is done in a similar way as in a public key infrastructure (PKI) where all servers and nodes register their public keys in a PKI bulletin Board (NB). For n servers and m edge nodes, pk_α and ϑk_i are the public keys of a server S_α and a node N_i respectively, where $pk_\alpha = g^{sk_\alpha}$ and $\vartheta k_i = g^{\kappa_i}$ such that sk_α and κ_i are the corresponding secret keys (unknown to the NB). The public parameters $\lambda, \mathbb{G}, \mathbb{G}_T, p, g, h, n, m, k, H$ are also published in the NB.
- *Local training*: For a set of m edge nodes, each node N_i trains a machine learning model M using its local data D_i with global parameter $\tilde{\omega}_{agg}$ received from the aggregators. In the first round, a random initial global parameter ω_0 is used. A new local parameter is computed as follows $\omega_i = \nabla L_f(D_i, M)$, where, ∇L_f is the derivative of the loss function L_f and $M = M - \eta\frac{\omega_{agg}}{n}$.
- *Verifiable authenticated sharing:* After obtaining a new local parameter ω_i, a blinding factor r_i is randomly chosen. Each node N_i runs an instance of the **VSS** protocol. The protocol starts by generating random coefficients $a_{ij} \in \{0,1\}^l$, such that $i \in \{1, \ldots, m\}$ and $j \in \{0, \ldots, k-1\}$, where $a_{i0} = \omega_i$. Then, for each coefficient a_{ij}, a random coefficient $b_{ij} \in \{0,1\}^l$ is generated to be used in proof construction to commit to the shares. Then, **VSS.Share** outputs a share $(\omega_{i\alpha}, r_{i\alpha})$ such that $i \in \{1, \ldots, m\}, \alpha \in \{1, \ldots, n\}$ is associated with the public proofs π_i to be published in the NB. After that, each node N_i runs a **BLS.Sign** instance to sign the shares with the node's private key. When a share is received, a server S_α verifies the signature and the correctness of the shares making use of the public proofs π_i. Therefore, the server S_α is able to verify that the coefficients used in the construction of the shares are correct. If they are not valid, the server S_α publishes a complaint against the node N_i along with the received share. When at least k complaints are received against a given node N_i, this node is banned from the system.
- *Verifiable secure aggregation*: To compute the aggregation of local parameters ω_i based on the distributed shares $\omega_{i\alpha}$, each server S_α will compute a partial aggregation using the correct shares. Suppose that servers possess d correct shares such that $1 < d \leq m$. Then, each server S_α computes $\tilde{\omega}_\alpha = \sum_{i=1}^{d+1} \omega_{i\alpha}$ and $\tilde{r}_\alpha = \sum_{i=1}^{d+1} r_{i\alpha}$. To reconstruct the global parameter, at least k partial aggregations are needed. Therefore, each server S_α sends his partial aggregations $\tilde{\omega}_\alpha, \tilde{r}_\alpha$ to the other servers along with an associated signature sig_α. Then, a server S_β verifies the signatures and the correctness of the received partial aggregations. If the verification is not successful, the server S_β will publish a complaint against the server S_α. At least k honest servers can reconstruct the global parameter $\tilde{\omega}_{agg}$, $\tilde{r}_{agg}$ using Lagrange polynomial interpolation. Then, a server S_β sends the global parameter $\tilde{\omega}_{agg}, \tilde{r}_{agg}$ along with a short signature $sigAgg_\beta$ to the d verified nodes.
- *Global model verification*: Each node receives $(\tilde{\omega}_{agg}, \tilde{r}_{agg})$ from at least k honest servers. A node N_i starts by verifying the short signature and the correctness of the global parameter $\tilde{\omega}_{agg}$ received from a server S_β. If the verification is not valid, N_i publishes a complaint against the server S_β and starts the verification of other received values. Upon a valid verification, N_i restarts from the local training phase using the new value $\tilde{\omega}_{agg}$. The process is to be repeated until model convergence.

VI. Security Analysis

In this section, we discuss the security properties of the VeSAFL scheme, with a focus on three key requirements: correctness, verifiability, and privacy. Our analysis assesses the system's ability to perform its intended functions accurately and reliably. Then, validate that all exchanged information is verified by parties while ensuring the privacy of each node's local parameter, in the presence of malicious adversaries.

A. Correctness

The correctness analysis is composed of two parts. The correctness of partial aggregation (server side) and the correctness of the global aggregated parameter (node side).

Authorized licensed use limited to: Anhui University. Downloaded on October 13,2026 at 08:43:08 UTC from IEEE Xplore. Restrictions apply.

- **Step 0** *(Initialization):*

  Public Bulletin Board **NB**:

  * Register public keys pk_α for n servers and and ϑk_i for m edge nodes.
  * Publish the public parameters $\lambda, \mathbb{G}, \mathbb{G}_T, p, n, m, k, g, h, H$, $\{pk_\alpha\}_{\alpha=\{1,...,n\}}$, and $\{\vartheta k_i\}_{i=\{1,...,m\}}$

- **Step 1** *(Local training):*

  Edge Node N_i for $i \in \{1, ..., m\}$:

  * trains a local model m using local data D_i and the global parameter $\tilde{\omega}_{agg}$:

  $$M = M - \eta \frac{\tilde{\omega}_{agg}}{m},$$

  and computes a new local parameter as $\omega_i = \nabla L_f(D_i, M)$,

- **Step 2** *(Verifiable authenticated sharing):*

  Edge Node N_i for $i \in \{1, ..., m\}$:

  * randomly chooses r_i, outputs shares and proofs: $\pi_i, \{\omega_{i\alpha}, r_{i\alpha}\}_{\alpha=\{1,...,n\}} \leftarrow$ **VSS.Share**(ω_i, r_i)
  * sign the shares: $sig_{i\alpha} \leftarrow$ **BLS.Sign**$(\kappa_i, \omega_{i\alpha}, r_{i\alpha})$
  * sends privately $(sig_{i\alpha}, \omega_{i,\alpha}, r_{i,\alpha})$ to each server S_α
  * publishes π_i in NB.

  Server S_α for $\alpha \in \{1, ..., n\}$:

  * gets $\pi_i = \{\pi_{i,j}\}_{j\in\{0,...,k-1\}}$ and ϑk_i from NB
  * verifies the received signature from N_i using **BLS.Verify**$(\vartheta k_i, sig_{i\alpha}, \omega_{i\alpha}, r_{i\alpha})$
  * If the signature is valid, the server verifies the correctness of the received share using **VSS.Verify**$(\omega_{i\alpha}, r_{i\alpha}, \pi_i)$ If verification is not valid the server publishes a complaint against N_i. A node is removed from the system if he is a subject of at least k complaints. Otherwise server S_α proceeds to the next step.

- **Step 3** *(Verifiable secure aggregation):*

  Server S_α for $\alpha \in \{1, ..., n\}$:

  * computes a partial aggregations from d valid shares: $\tilde{\omega}_\alpha, \tilde{r}_\alpha \leftarrow$ **VSS.Agg**$(\{\omega_{i\alpha}, r_{i\alpha}\}_{i=\{1,...,d+1\}})$
  * computes $sig_\alpha \leftarrow$ **BLS.Sign**$(sk_\alpha, \tilde{\omega}_\alpha, \tilde{r}_\alpha)$
  * Sends the partial aggregations along with the signature $(sig_\alpha, \tilde{\omega}_\alpha, \tilde{r}_\alpha)$ privately to the other servers.

  Server S_β for $\beta \in \{1, ..., n\}$:

  * gets $\pi_i = \{\pi_{i,j}\}_{j\in\{0,...,k-1\}}$ and pk_α from NB
  * verifies the signature received from S_α using **BLS.Verif**$(pk_\alpha, sig_\alpha, \tilde{\omega}_\alpha, \tilde{r}_\alpha)$.
  * If signature is valid, server S_β verifies the correctness of the partial aggregated parameter received:

  $$e(\prod_{i=1}^{d+1}\prod_{l=0}^{k-1} \pi_{il}^{\alpha^l}, g) == e(g^{\tilde{\omega}_\alpha} h^{\tilde{r}_\alpha}, g). \quad (1)$$

  If verification is not valid, S_β publishes a complaint against S_α, if a server is subject to k complaints he is removed from the system. Otherwise S_β proceeds to the next step.
  * reconstructs the global aggregated parameter and the global random factor when at least k partial aggregations are proven to be valid: $(\tilde{\omega}_{agg}, \tilde{r}_{agg}) \leftarrow$ **VSS.Recon**$(\{\tilde{\omega}_\alpha, \tilde{r}_\alpha\}_{\alpha\in\{1,...,k+1\}})$.
  * computes $sigAgg_\alpha \leftarrow$ **BLSS.Sign**$(sk_\beta, \tilde{\omega}_{agg}, \tilde{r}_{agg})$
  * sends $(sigAgg_\beta, \tilde{\omega}_{agg}, \tilde{r}_{agg})$ to valid nodes.

- **Step 4** *(Global model verification):*

  Node N_i for $i \in \{1, ..., m\}$:

  * gets $\pi_i = \{\pi_{i,j}\}_{j\in\{0,...,k-1\}}$ and pk_β from NB
  * verifies the received signature using: **BLS.Verify**$(pk_\beta, sigAgg_\beta, \tilde{\omega}_{agg}, \tilde{r}_{agg})$.
  * If signature is valid, node N_i verifies the correctness of $\tilde{\omega}_{agg}$ and $\tilde{r}_{agg}$):

  $$e(\prod_{i=1}^{d+1} \pi_{i0}, g) == e(g^{\tilde{\omega}_{agg}} h^{\tilde{r}_{agg}}, g). \quad (2)$$

  If the verification is not valid, N_i publishes a complaint against the S_α and starts the verification of other received values. Upon a valid verification N_i restart from step 1 using the new value $\tilde{\omega}_{agg}$.

Fig. 3. Detailed processes of VeSAFL.

Authorized licensed use limited to: Anhui University. Downloaded on October 13,2026 at 08:43:08 UTC from IEEE Xplore. Restrictions apply.

Theorem 3: If d nodes follow the scheme, each honest server S_α can correctly compute the partial aggregation $\tilde{\omega}_\alpha, \tilde{r}_\alpha$.

Proof: If d nodes execute the sharing protocol correctly, the correct partial aggregation can be obtained by servers from the correct shares $\omega_{i\alpha}, r_{i\alpha}$. According to Algorithm 4,

$$\tilde{\omega}_\alpha = \sum_{i=1}^{d+1} \omega_{i\alpha},$$

$$\tilde{r}_\alpha = \sum_{i=1}^{d+1} r_{i\alpha}.$$

□

Theorem 4: If at least k servers are honest, the global aggregations can be reconstructed correctly.

Proof: If at least k servers are honest, the global aggregations $\tilde{\omega}_{agg}, \tilde{r}_{agg}$ can be reconstructed using the k correct partial aggregations $\tilde{\omega}_\alpha, r_\alpha$, following Algorithm 5,

$$\begin{aligned}\tilde{\omega}_{agg} &= \sum_{\alpha=1}^{k+1} \tilde{\omega}_\alpha \lambda_\alpha = \sum_{\alpha=1}^{k+1}\sum_{i=1}^{d+1} \tilde{\omega}_{i\alpha}\lambda_\alpha \\ &= \sum_{i=1}^{d+1} f_i(0) = \sum_{i=1}^{d+1} \omega_i, \\ \tilde{r}_{agg} &= \sum_{\alpha=1}^{k+1} \tilde{r}_\alpha \lambda_\alpha = \sum_{\alpha=1}^{k+1}\sum_{i=1}^{d+1} \tilde{r}_{i\alpha}\lambda_\alpha \\ &= \sum_{i=1}^{d+1} g_i(0) = \sum_{i=1}^{d+1} r_i.\end{aligned}$$

□

B. Verifiability

The verifiability property covers three scenarios: - Nodes that can communicate forged shares in the sharing process. - Servers that may forge the partial aggregation. And - The global aggregated parameter can be falsified by servers. Our scheme gives the possibility to verify the communicated information at each step.

Theorem 5: Given published proofs π_i. A server can verify the integrity of a received share $(\omega_{i\alpha}, r_{i\alpha})$ if Algorithm 3 holds.

Proof: Given a set of proofs $\pi_i = \{\pi_{i,j}\}_{j=\{0,\ldots,k-1\}}$ and a share $(\omega_{i\alpha}, r_{i\alpha})$, the Algorithm 3 verifies that the share is computed using the committed coefficients.

$$\begin{aligned}e\left(\prod_{l=0}^{k-1} \pi_{i,l}^{\alpha^l}, g\right) &= e\left(\prod_{l=0}^{k-1} g^{a_{il}^{\alpha^l}} h^{b_{il}^{\alpha^l}}, g\right) \\ &= e\left(g^{a_{i0}+\cdots+a_{ik-1}\alpha^{k-1}} h^{b_{i0}+\cdots+b_{ik-1}\alpha^{k-1}}, g\right) \\ &= e\left(g^{\omega_{i\alpha}} h^{r_{i\alpha}}, g\right).\end{aligned}$$

□

For the remaining theorems, we assume that each server is in possession of d correctly verified shares.

Theorem 6: Given published proofs π_i. Each server S_α can verify the integrity of the partial aggregation $(\tilde{\omega}_\alpha, \tilde{r}_\alpha)$ received from other servers if the (1)in Fig. 3 holds.

Proof: Given a set of proofs $\pi_i = \{\pi_{ij}\}_{j=\{0,\ldots,k-1\}}$ and a partial aggregation $(\tilde{\omega}_\alpha, \tilde{r}_\alpha)$, the (1) verifies that the partial aggregation is computed using the correct shares.

$$\begin{aligned}e\left(\prod_{i=1}^{d+1}\prod_{l=0}^{k-1} \pi_{il}^{\alpha^l}, g\right) &= e\left(\prod_{i=1}^{d+1} g^{\omega_{i\alpha}} h^{r_{i\alpha}}, g\right) \\ &= e\left(g^{\tilde{\omega}_\alpha} h^{\tilde{r}_\alpha}, g\right).\end{aligned}$$

□

Theorem 7: The integrity of the global aggregated parameter is verified if (2) in Fig. 3 holds.

Proof: Given a set of proofs $\pi_i = \{\pi_{ij}\}_{j=\{0,\ldots,k-1\}}$ and the global aggregation $(\tilde{\omega}_{agg}, \tilde{r}_{agg})$, the (2) verifies that the global aggregation is computed using the correct partial aggregations.

$$\begin{aligned}e\left(\prod_{i=1}^{d+1} \pi_{i,0}, g\right) &= e\left(\prod_{i=1}^{d+1} g^{a_{i0}} h^{b_{i0}}, g\right) = \prod_{i=1}^{d+1} e(g^{\omega_i} h^{r_i}, g) \\ &= e\left(g^{\sum_{i=1}^{d+1}\omega_i} h^{\sum_{i=1}^{d+1} r_i}, g\right) \\ &= e\left(g^{\tilde{\omega}_{agg}} h^{\tilde{r}_{agg}}, g\right).\end{aligned}$$

□

C. Privacy

Our scheme can ensure that the aggregation servers do not get local parameters. Meanwhile, we consider that the global aggregated model can only be obtained when at least k servers are honest. In our scheme, we aim to protect the local and global parameters against malicious attackers.

Theorem 8: In VeSAFL, a malicious adversary can never learn the node's local parameter ω_i. The global parameter cannot be leaked in the presence of at most $k-1$ malicious colluding servers.

Proof: The local parameter share is computed based on the Pedersen secret sharing scheme. Therefore, the protocol is information-theoretically secure. Thus, an active adversary never learns the local parameter independently from shares [23].

Without loss of generality, we assume that servers $S_1, \ldots, S_k$ $(k \leq n-2)$ collude.

Let

$$\tilde{\omega}'_{agg} = \sum_{\alpha=1}^{k} \lambda_\alpha \tilde{\omega}_\alpha, \qquad \tilde{r}'_{agg} = \sum_{\alpha=1}^{k} \lambda_\alpha \tilde{r}_\alpha$$

such that

$$\lambda_\alpha = \prod_{\substack{1\leq i\leq k \\ i\neq\alpha}} \frac{i}{i-\alpha}.$$

Let $View_{S_\alpha} = [\tilde{\omega}'_{agg}, \tilde{\omega}_\alpha, \alpha \in \{1, \ldots, k\}]$.

From Theorem 2, we conclude that the probability distribution of a server learning the global parameter giving $view_{S_\alpha}$ is the same as the probability distribution of a server learning the global parameter without any views.

$$Pr[S_\alpha \text{ has } \tilde{\omega}_{agg} | view_{S_\alpha}] = Pr[S_\alpha \text{ has } \tilde{\omega}_{agg}].$$

Hence, the knowledge of a server S_α colluding with at most $k-1$ where $k \leq n-2$ does not provide any additional information about the local and global parameters beyond what has already been published. □

Authorized licensed use limited to: Anhui University. Downloaded on October 13,2026 at 08:43:08 UTC from IEEE Xplore. Restrictions apply.

VII. Experimental Analysis

In this section, we present the performance analysis of the VeSAFL scheme. Extensive experiments have been conducted to evaluate the model's accuracy and the cost overhead. When evaluating VeSAFL, we focus on accuracy comparison against the baseline FedAVG [3], which represents the standard federated learning protocol without privacy or verifiability requirements. Note that VeSAFL preserves the same training logic and does not modify the model update semantics, ensuring mostly similar learning convergence behavior.

While several prior works such as [9], [32], [33], [35] also provide privacy-preserving or verifiable FL schemes, their cryptographic foundations (e.g., homomorphic encryption, centralized assumptions, or system-specific primitives) differ significantly. Due to these underlying architectural and implementation differences, reproducing their schemes within our decentralized, multi-server environment is not trivial and could introduce unfairness in empirical accuracy comparisons. Therefore, we provide an efficiency cost analysis, evaluating their computational and communication overheads under their respective security models.

A. Efficiency Cost Analysis

We analyze the efficiency of the VeSAFL protocol in terms of computation, communication, and storage overhead, for both nodes and servers. The local training phase is excluded from this analysis, as it remains the same as in standard federated learning. Let us n denote the number of servers, m the number of participating nodes, v the size of the model vector, and s the size of cryptographic materials such as BLS signatures and proofs. For computation cost, we overload the notation s to also represent the cost of performing expensive cryptographic operations, such as pairing or exponentiations.

a) Node-Side Costs: Computation Cost. Each node shares its local model vector of size v by generating and evaluating two polynomials at n points, resulting in $\mathcal{O}(n \cdot v)$ scalar operations. It then produces n BLS signatures, one per server, to ensure authenticity of the shared values. The cryptographic cost associated with generating each signature is $\mathcal{O}(s)$. Therefore, the total computation cost at the node side is: $\mathcal{O}(n(v+s))$.

Communication Cost: Each node sends one share of its local model to each of the n servers, along with a corresponding BLS signature and publish the proof for each share in the NB. The transmitted data therefore includes the full model vector (split across shares) and n cryptographic materials of size s, giving a communication cost of: $\mathcal{O}(v + n \cdot s)$.

Storage Cost: Each node stores its private key and its local model vector of size v. All other values (e.g., shares, signatures) are generated per round and not persistently stored. Thus, node-side storage is: $\mathcal{O}(v)$.

b) Server-Side Costs: Computation Cost. Each server verifies m shares using signatures and proofs verifications, resulting in $\mathcal{O}(m \cdot s)$ cryptographic operations. It also verifies partial aggregates from $n-1$ peer servers ($\mathcal{O}(n \cdot s)$) and reconstructs the global model vector via Lagrange interpolation over v elements, contributing $\mathcal{O}(v)$ additional cost. Thus, the total computation cost at the server side is: $\mathcal{O}((m+n) \cdot s + v)$.

Communication Cost: Each server receives one share from each of the m nodes, each accompanied by a signature and proof of size s, incurring a cost of $\mathcal{O}(m \cdot s)$. It also sends its partial aggregate to each of the other $n-1$ servers ($\mathcal{O}(n \cdot s)$) and broadcasts the final global model vector ($\mathcal{O}(v)$). Thus, the total server-side communication cost is: $\mathcal{O}((m+n) \cdot s + v)$.

Storage Cost: Each server stores its private key, the reconstructed global model vector of size v, and the m received shares. These cryptographic materials have a cumulative size of $\mathcal{O}(m \cdot s)$, leading to a total storage cost of: $\mathcal{O}(v + m \cdot s)$.

c) Comparison: We compare our VeSAFL protocol with the scheme proposed by Wang et al. [35], which follows a single-server architecture. We also include a comparison with VerifyNet [29], which extends the secure aggregation protocol of Bonawitz et al. [9] with additional verifiability guarantees. Since VerifyNet builds directly on Bonawitz's protocol, its asymptotic efficiency is largely inherited, with extra cryptographic operations added for shares and aggregates verification. For consistency, we unify the notation across all schemes as follows: n denotes the number of servers (in VeSAFL's multi-server setting), m the number of clients, v the size of the model vector, and s the size or computational cost of cryptographic operations such as BLS signatures and proofs, and k any additional setup or masking operations. The asymptotic costs for clients and servers are summarized in Tables III and IV.

TABLE III
Asymptotic Cost Comparison (Client/Node Side)

Scheme	Computation	Communication	Storage
[30]	$\mathcal{O}(vm^2 + mv)$	$\mathcal{O}(v+m)$	$\mathcal{O}(v+m)$
[36]	$\mathcal{O}(v+k+s)$	$\mathcal{O}(v+s)$	$\mathcal{O}(v+s)$
VeSAFL	$\mathcal{O}(n(v+s))$	$\mathcal{O}(v+ns)$	$\mathcal{O}(v)$

TABLE IV
Asymptotic Cost Comparison (Server Side)

Scheme	Computation	Communication	Storage
[30]	$\mathcal{O}(m^2 v)$	$\mathcal{O}(m^2 + mv)$	$\mathcal{O}(m^2 + v)$
[36]	$\mathcal{O}(mv + k + ms)$	$\mathcal{O}(ms + v)$	$\mathcal{O}(v + ms)$
VeSAFL	$\mathcal{O}((m+n)s + v)$	$\mathcal{O}((m+n)s + v)$	$\mathcal{O}(v + ms)$

In summary, VeSAFL introduces additional computation and communication overhead at the node side due to its multi-server verifiable design, but offers enhanced robustness and decentralization. In contrast, Wang's scheme is lighter for clients and achieves efficiency through a single-server architecture, while relying on symmetric homomorphic encryption and precomputed signature aggregation to ensure privacy and correctness. Compared to centralized federated learning aggregation schemes, VeSAFL avoids single point of failure by distributing trust across multiple servers and can tolerate up to $n-k$ malicious or offline participants. Although pairing operations and other cryptographic primitives used in verification are relatively expensive, they are limited to specific phases and can be efficiently implemented using cryptographic libraries such as Charm library [26].

Authorized licensed use limited to: Anhui University. Downloaded on October 13,2026 at 08:43:08 UTC from IEEE Xplore. Restrictions apply.

TABLE V
DATASETS DESCRIPTION

Dataset	Type	# Classes	Partitioning	Input Size
MNIST [55]	Digits (0–9)	10	IID / Non-IID (label skew)	28×28 Grayscale
FEMNIST [54]	Handwriting (digits + letters)	62	Non-IID (by writer ID)	28×28 Grayscale
CIFAR-10 [56]	Object images	10	IID / Non-IID (Dirichlet)	32×32 RGB

B. Experimental Setup

The experiments were run on MacOS 10.15.7 over a MacBookPro (2018) with processor Intel i5 CPU@2.3 GHz, with 8 GB LPDDR3 2133 MHz RAM. VeSAFL was implemented in Python 3.7.7 where the cryptographic primitives are conducted using Charm library for pairing-based cryptosystem [26]. The cryptographic components of VeSAFL were implemented using the Charm-Crypto library. For secret sharing and commitment, we used a 256-bit prime-order group. The global model verification via short signatures was instantiated using a pairing-based elliptic curve group providing 128-bit security. These parameters remained fixed across all experiments to ensure consistency in evaluation. While stronger cryptographic settings would increase execution time, particularly for pairing-based operations, they do not affect the accuracy or convergence of the model training process.

The FL setup is simulated using Flower Framework [27] to simulate the nodes and the aggregators. We emphasize that the use of Flower serves as a simulation scaffold to emulate the FL environment. Our focus lies in evaluating the cryptographic layer introduced by VeSAFL, which is designed to be agnostic to the underlying FL framework. In practice, the proposed scheme can be integrated with any standard FL infrastructure, without being tied to a specific orchestration platform. VeSAFL is evaluated on three datasets: MNIST [53], FEMNIST, and CIFAR-10, each offering varying complexity and distribution heterogeneity:

- MNIST [53] contains grayscale digit images (28×28 pixels).
- FEMNIST (part of the LEAF benchmark [52]) contains 62 classes including digits and upper/lowercase letters.
- CIFAR-10 [54] contains RGB images (32×32 pixels) across 10 object classes, suitable for cross-device FL scenarios.

The partitioning methods are chosen to simulate practical non-IID FL conditions. For the IID setting, the data is randomly shuffled and equally split across all clients. For non-IID scenarios:

- MNIST is partitioned using label skew, where each client receives data from only a small subset of classes.
- FEMNIST is inherently non-IID, as its data is naturally grouped by writer identity.
- CIFAR-10 is partitioned using a Dirichlet distribution with $\alpha = 0.1$, a widely used method to control label distribution heterogeneity across clients.

Table V provides a summary of each dataset's characteristics.

C. Model Accuracy

VeSAFL is simulated with respect to m edge nodes and n aggregators with a threshold k. In theorem 1, it has been proven that, if at least k aggregators are honest the global parameter is always reconstructed correctly, thus we test our VeSAFL accuracy with $n = 5, k = 3, m = 100$. We evaluate our VeSAFL and the baseline FedAVG [3] using a convolution neural network (CNN) under the aforementioned IID and non-IID databases. The CNN network consists of two 5×5 convolutional layers (the first with 32 channels, the second with 64, each followed with 2×2 max pooling), a fully connected layer with 512 units and ReLu activation, and a final softmax output layer. In the baseline FL model [3], a fraction of 10% of nodes is selected in each round to participate in the model update. Then, the nodes upload plain local updates directly, where the aggregation server directly computes the global model update using federated averaging (FedAVG). However, in our VeSAFL scheme, the nodes upload only shares of the local updates. Then, when at least k are servers honest, they can reconstruct the global model update.

Accuracy comparisons between VeSAFL and FedAVG are conducted over 400 communication rounds where $n = 5,\ k = 3,\ m = 100$. As shown in Figs. 4, 5, 6, our scheme achieves comparable accuracy on all datasets while preserving privacy and ensuring verifiable correctness of local and global updates.

It is important to note that the aggregation process in VeSAFL maintains the same mathematical structure as standard federated averaging. Our contribution lies in securely executing this process using verifiable secret sharing and cryptographic proofs, ensuring correctness even in adversarial settings. As confirmed in our experimental results, this design achieves equivalent model accuracy while enforcing strong privacy and verifiability guarantees.

D. Cost Analysis

Considering n servers and m nodes, each node communicates $n.m$ shares ω_{ij} and $k.m$ proofs π_i. Whereas the aggregators have a constant global output $\tilde{\omega}_{agg}$, and during the aggregation, they are communicating m partial aggregations. In Fig. 7(a) we can notice that the computational overhead of nodes is consistently increasing linearly when increasing the number of servers. While, when increasing the number of nodes, the computational cost is not significantly impacted. This arises from the independence of each node's execution from the other nodes. On the other hand, the communication overhead of a single node increases when increasing the number of servers as shown in Fig. 7(b). This is normal since the amount of authenticated shares and proofs that need to be communicated is increased when the number of servers is increased.

Fig. 8(a) depicts the computational overhead of servers when increasing nodes. As expected, the execution time is linearly increasing when increasing the number of nodes, since the

Authorized licensed use limited to: Anhui University. Downloaded on October 13,2026 at 08:43:08 UTC from IEEE Xplore. Restrictions apply.

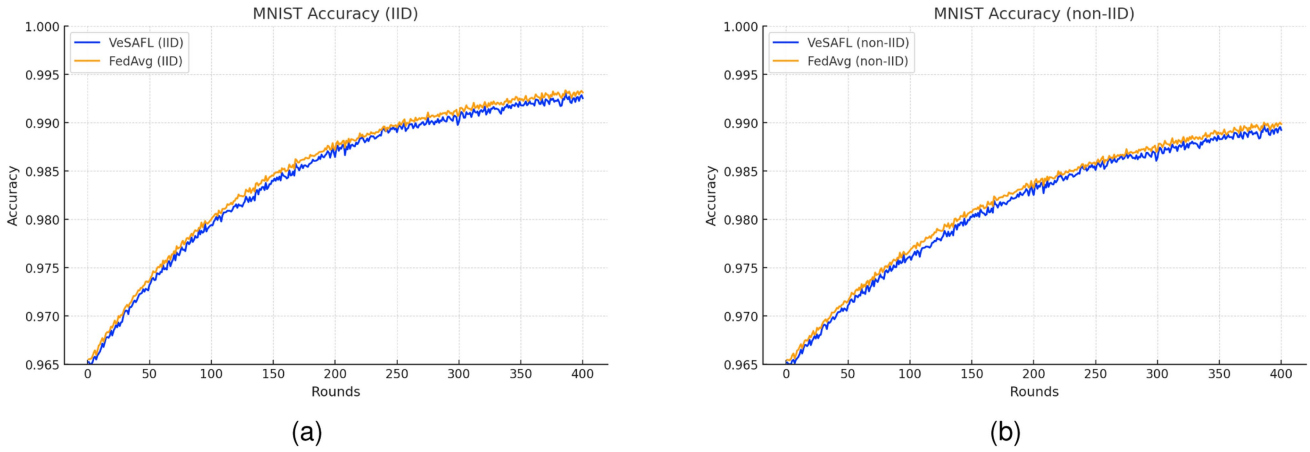


Fig. 4. CNN model accuracy comparison between FedAVG and VeSAFL for 400 rounds on MNIST (a) IID data distribution (b) non-IID data distribution.

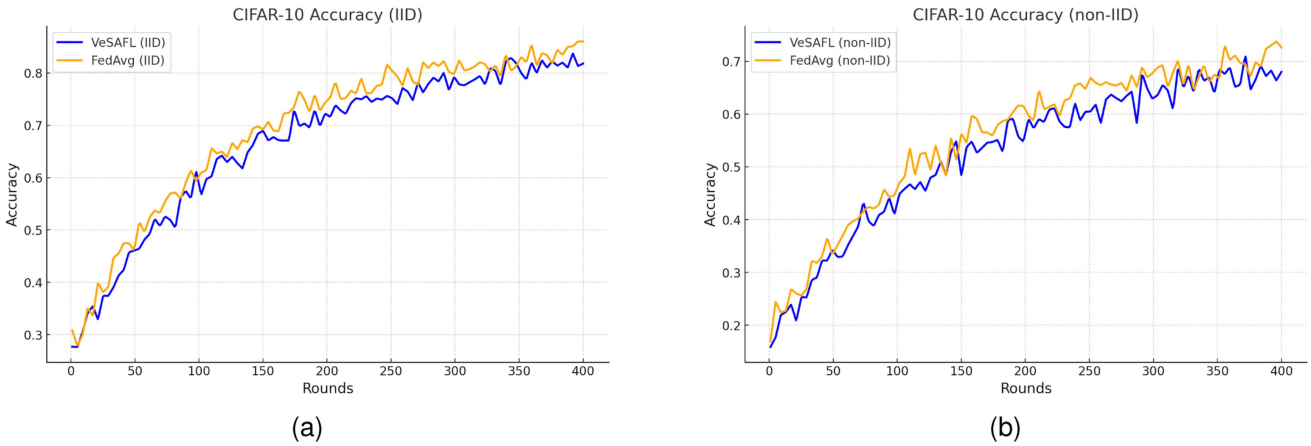


Fig. 5. CNN model accuracy comparison between FedAVG and VeSAFL for 400 rounds on CIFAR-10 (a) IID data distribution (b) non-IID data distribution.

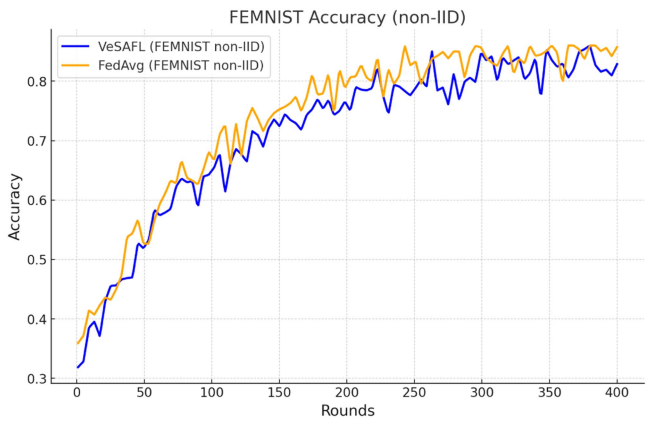


Fig. 6. CNN model accuracy comparison between FedAVG and VeSAFL for 400 rounds on FEMNIST.

number of proofs and signatures to be verified is also increasing. However, we can notice that when increasing the number of servers our VeSAFL is efficient since the reconstruction and short signature algorithms are efficient even when increasing the number of servers. As for the communication overhead, the servers need to exchange partial and global aggregations which directly depends on the number of nodes and servers. As shown in Fig. 8(b), the communication overhead is linearly dependent on the number of nodes and servers. In addition, we evaluate the computational overhead of the additional cryptographic primitives used in the different stages: sharing, signing, and verification. Fig. 9 depicts the sharing, reconstruction, proof generation, and verification running time when increasing the number of gradient parameters, the number of nodes (Fig. 9(a)), and the number of aggregation servers (Fig. 9(b)).

To protect the scheme against active adversaries, all participants sign the exchanged information. Fig. 10 shows the running time of the signature generation and verification processes when increasing the number of parameters along with the nodes (Fig. 10(a)) and the aggregation servers (Fig. 10(b)). From the previous figures, we observe that the running time of both the verifiable secret sharing the bilinear short signature increases slightly with the increasing number of model gradient parameters while increasing linearly when increasing both the number of nodes and servers. The main reason is that the

Authorized licensed use limited to: Anhui University. Downloaded on October 13,2026 at 08:43:08 UTC from IEEE Xplore. Restrictions apply.

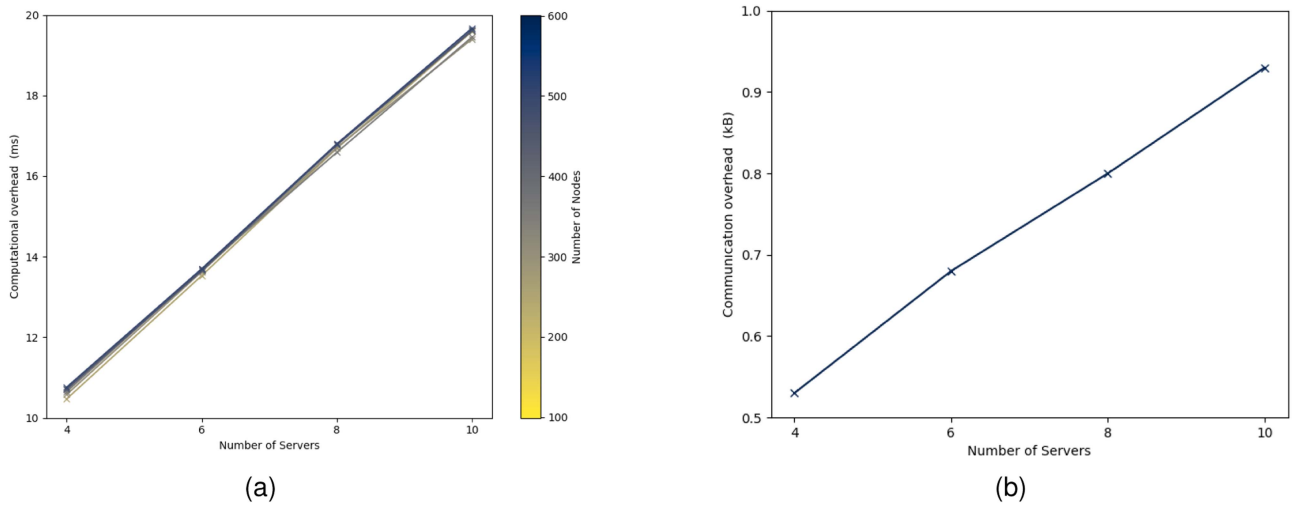


Fig. 7. Cost analysis of different numbers of nodes with different numbers of servers. (a) Computational overhead (b) Communication overhead.

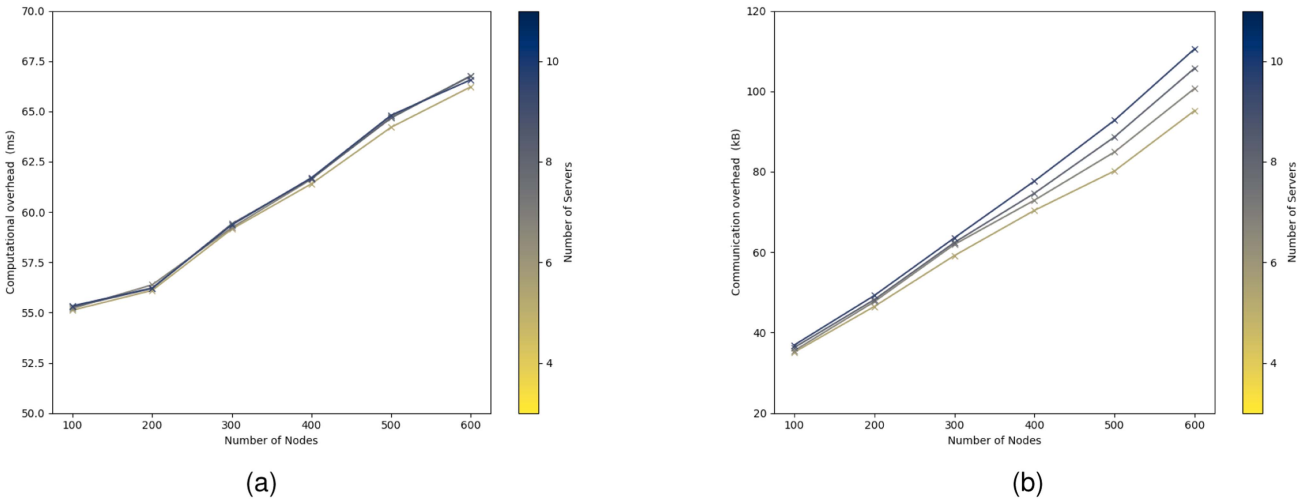


Fig. 8. Cost analysis of different numbers of servers with different numbers of nodes. (a) Computational overhead (b) Communication overhead.

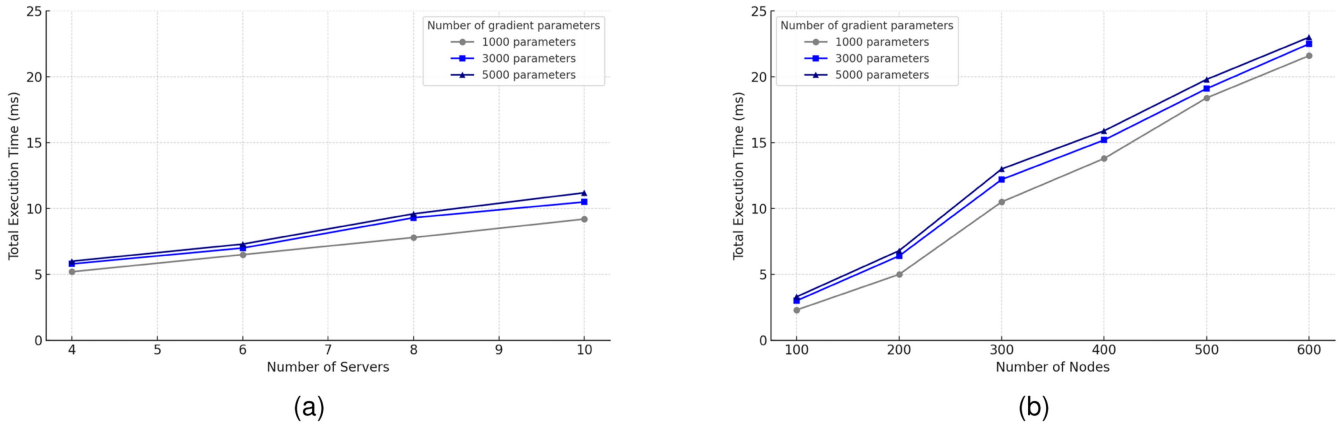


Fig. 9. Running time of the verifiable secret sharing (a) for a different number of gradient parameters and servers with $m = 100$ (b) for a different number of gradient parameters and nodes with $n = 4$.

Authorized licensed use limited to: Anhui University. Downloaded on October 13,2026 at 08:43:08 UTC from IEEE Xplore. Restrictions apply.

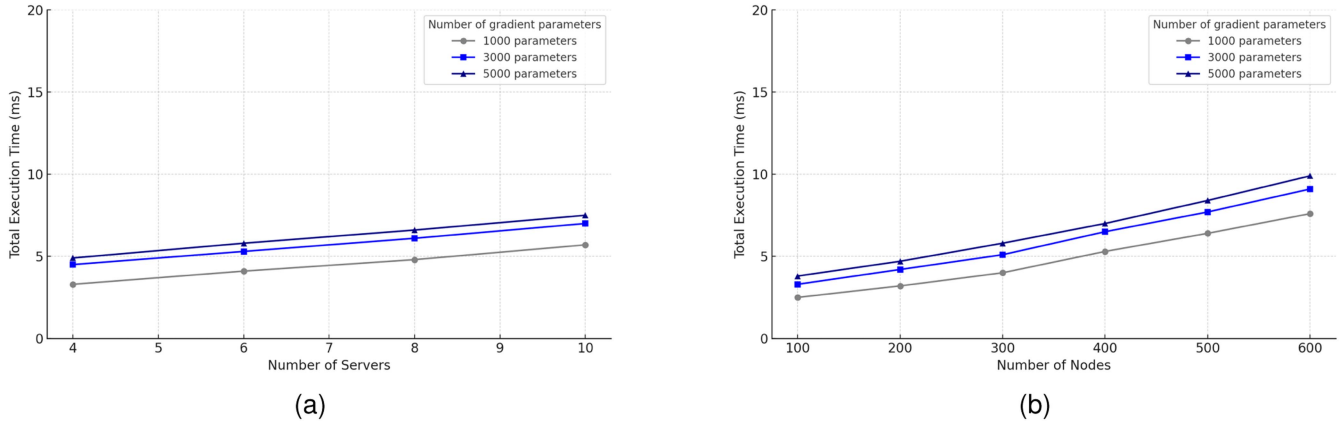


Fig. 10. Running time of the bilinear short signature (a) for a different number of gradient parameters and servers with $m = 100$ (b) for a different number of gradient parameters and nodes with $n = 4$.

computation cost of the nodes mainly comes from the cryptographic primitives which are proportional to the number of nodes and servers.

VIII. Discussion and Future Directions

This section reflects on VeSAFL's current security properties, its limitations, and potential avenues for future enhancement. VeSAFL preserves the correctness of global aggregation through cryptographic commitments and verifiable sharing while maintaining the same convergence behavior as standard federated averaging. The cryptographic aggregation layer ensures that local updates remain hidden during transmission and computation, while the final global model is identical to that obtained through unprotected averaging. This guarantees that security and verifiability do not impact model accuracy.

The scheme assumes a reliable, publicly accessible bulletin board, which ensures that commitments and verification proofs are immutable and auditable throughout protocol execution (as detailed in Section IV). These properties are central to guaranteeing the soundness of complaint resolution and global model verification. Even if a malicious node colludes with a subset of servers, it gains no advantage unless the number of corrupted aggregators exceeds the reconstruction threshold k. Any attempt to bypass verifiability will be detected by an honest verifier, either a node or an aggregator.

VeSAFL's complaint mechanism enforces accountability through threshold-triggered exclusion. A node that accumulates k valid complaints is prevented from accessing future model updates, limiting its ability to mount repeated inference attacks. However, this defense is partial and does not eliminate inference risks entirely. Differential privacy [55], randomized model dissemination, or cryptographically enforced output obfuscation are promising additions to provide stronger mitigation. Recent approaches, such as those proposed by Fotohi et al. [4], [40], demonstrate how techniques like blockchain integration and lightweight cryptographic frameworks can enhance traceability and privacy in adversarial FL settings. Integrating such techniques with VeSAFL's verifiable architecture is a compelling direction for future work.

The current design does not verify the semantic correctness of model updates. Clients may submit syntactically valid yet adversarial gradients that pass all cryptographic checks but degrade model performance or embed backdoors. These poisoning threats, including Sybil-based manipulation, are not addressed in VeSAFL and are consistent with assumptions made in prior work such as VerSA [32] and VPFL [33]. Robust aggregation strategies, anomaly detection, or trust-based client filtering could be integrated into future extensions to improve resilience [56], [57], [58].

Scalability remains a practical concern. While our cost analysis shows linear growth in communication and verification overheads, deploying VeSAFL in networks with thousands of participants may require architectural improvements. The decentralized multi-aggregator setup helps mitigate central bottlenecks, but further enhancements such as hierarchical aggregation, adaptive load distribution, or edge-aware deployment models could improve efficiency under dynamic participation and heterogeneous connectivity.

Finally, privacy-preserving public auditability remains an open research problem. While VeSAFL ensures that complaints are verifiable through published commitments, doing so without revealing sensitive data to third-party auditors is not yet supported. Incorporating techniques such as zero-knowledge proofs or threshold decryption could enable complaint resolution and auditability while preserving confidentiality.

Future directions include: (i) incorporating differential privacy to limit inference risks; (ii) supporting poisoning-resistant model aggregation; (iii) extending the scheme to asynchronous or hierarchical settings for greater scalability; (iv) mitigating Sybil threats through identity validation or reputation tracking; and (v) enhancing complaint verification using privacy-preserving cryptographic tools. These improvements aim to reinforce VeSAFL's security, privacy, and practicality for deployment in adversarial federated learning environments.

IX. Conclusion

In this paper, we introduced VeSAFL, a novel scheme designed to enable verifiable, privacy-preserving FL within edge

Authorized licensed use limited to: Anhui University. Downloaded on October 13,2026 at 08:43:08 UTC from IEEE Xplore. Restrictions apply.

computing environments. Our approach addresses the key challenges of privacy, verifiability, and decentralization by distributing model updates across edge nodes, reducing the dependency on centralized cloud servers. VeSAFL leverages advanced cryptographic techniques, including verifiable secret sharing and bilinear short signatures, to ensure the correctness, verifiability, and privacy of the model updates. Through experimental evaluations, we demonstrated the scheme's efficiency while maintaining robust privacy guarantees. In addition, our security analysis has confirmed that VeSAFL is resilient against active adversaries, demonstrating its capacity to safeguard data privacy and integrity in the defined threat scenarios. Future research directions include exploring the integration of other privacy methods such as differential privacy and the deployment of VeSAFL in real world scenarios.

REFERENCES

[1] Z. Li, V. Sharma, and S. Mohanty, "Preserving data privacy via federated learning: Challenges and solutions," *IEEE Consum. Electron. Mag.*, vol. 9, no. 3, pp. 8–16, May 2020.

[2] J. Konečnỳ, H. McMahan, D. Ramage, and P. Richtárik, "Federated optimization: Distributed machine learning for on-device intelligence," 2016, *arXiv:1610.02527*.

[3] B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. Arcas, "Communication-efficient learning of deep networks from decentralized data," in *Proc. Int. Conf. Artif. Intell. Statist.*, 2017, pp. 1273–1282.

[4] R. Fotohi, F. Aliee, and B. Farahani, "A lightweight and secure deep learning model for privacy-preserving federated learning in intelligent enterprises," *IEEE Internet Things J.*, vol. 11, no. 19, pp. 31988–31998, Oct. 2024.

[5] Q. Yang, Y. Liu, T. Chen, and Y. Tong, "Federated machine learning: Concept and applications," *ACM Trans. Intell. Syst. Technol.*, vol. 10, pp. 1–19, 2019.

[6] K. Bonawitz et al., "Towards federated learning at scale: System design," in *Proc. Mach. Learn. Syst.*, vol. 1, pp. 374–388, 2019.

[7] J. Bouamama, Y. Benkaouz, and M. Ouzzif, "EdgeSA: Secure aggregation for privacy-preserving federated learning in edge computing," in *Proc. IEEE Int. Conf. Dependable Autonomic Secure Comput.*, 2023, pp. 0375–0382.

[8] P. Mohassel and Y. Zhang, "SecureML: A system for scalable privacy-preserving machine learning," in *Proc. IEEE Symp. Secur. Privacy*, 2017, pp. 19–38.

[9] K. Bonawitz et al., "Practical secure aggregation for privacy-preserving machine learning," in *Proc. 2017 ACM SIGSAC Conf. Comput. Commun. Secur.*, 2017, pp. 1175–1191.

[10] S. Truex et al., "A hybrid approach to privacy-preserving federated learning," in *Proc. 12th ACM Workshop Artif. Intell. Secur.*, 2019, pp. 1–11.

[11] H. McMahan, D. Ramage, K. Talwar, and L. Zhang, "Learning differentially private recurrent language models," 2017, *arXiv: 1710.06963*.

[12] K. Wei et al., "Federated learning with differential privacy: Algorithms and performance analysis," *IEEE Trans. Inf. Forensics Secur.*, vol. 15, pp. 3454–3469, 2020.

[13] C. Gentry, "Fully homomorphic encryption using ideal lattices," in *Proc. 41st Annu. ACM Symp. Theory Comput.*, 2009, pp. 169–178.

[14] C. Dwork et al., "The algorithmic foundations of differential privacy," *Found. Trends Theor. Comput. Sci.*, vol. 9, pp. 211–407, 2014.

[15] W. Lim et al., "Federated learning in mobile edge networks: A comprehensive survey," *IEEE Commun. Surveys Tuts.*, vol. 22, no. 3, pp. 2031–2063, Third Quarter 2020.

[16] P. Kairouz et al., "Advances and open problems in federated learning," *Found. Trends Mach. Learn.*, vol. 14, 2021, Art. no. 210.

[17] A. Blanco-Justicia, J. Domingo-Ferrer, S. Martınez, D. Sanchez, A. Flanagan, and K. Tan, "Achieving security and privacy in federated learning systems: Survey, research challenges and future directions," *Eng. Appl. Artif. Intell.*, vol. 106, 2021, Art. no. 104468.

[18] A. Brecko, E. Kajati, J. Koziorek, and I. Zolotova, "Federated learning for edge computing: A survey," *Appl. Sci.*, vol. 12, 2022, Art. no. 9124.

[19] Y. Zhao, M. Li, L. Lai, N. Suda, D. Civin, and V. Chandra, "Federated learning with non-IID data," 2018, *arXiv: 1806.00582*.

[20] A. Shamir, "How to share a secret," *Commun. ACM*, vol. 22, pp. 612–613, 1979.

[21] G. Blakley, "Safeguarding cryptographic keys," in *Proc. Int. Workshop Manag. Requirements Knowl.*, 1979, pp. 313–313.

[22] C. Asmuth and J. Bloom, "A modular approach to key safeguarding," *IEEE Trans. Inf. Theory*, vol. 29, no. 2, pp. 208–210, Mar. 1983.

[23] T. Pedersen, "Non-interactive and information-theoretic secure verifiable secret sharing," in *Proc. Annu. Int. Cryptol. Conf.*, 1991, pp. 129–140.

[24] D. Boneh, B. Lynn, and H. Shacham, "Short signatures from the Weil pairing," in *Proc. Int. Conf. Theory Appl. Cryptol. Inf. Secur.*, 2001, pp. 514–532.

[25] D. Boneh and X. Boyen, "Short signatures without random oracles," in *Proc. Int. Conf. Theory Appl. Cryptographic Techn.*, 2004, pp. 56–73.

[26] J. Akinyele et al., "Charm: A framework for rapidly prototyping cryptosystems," *J. Cryptographic Eng.*, vol. 3, pp. 111–128, 2013.

[27] D. Beutel et al., "Flower: A friendly federated learning framework," 2022, *arXiv: 2007.14390*.

[28] A. Chandramouli, A. Choudhury, and A. Patra, "A survey on perfectly secure verifiable secret-sharing," *ACM Comput. Surv.*, vol. 54, pp. 1–36, 2022.

[29] G. Xu, H. Li, S. Liu, K. Yang, and X. Lin, "VerifyNet: Secure and verifiable federated learning," *IEEE Trans. Inf. Forensics Secur.*, vol. 15, pp. 911–926, 2020.

[30] G. Han, T. Zhang, Y. Zhang, G. Xu, J. Sun, and J. Cao, "Verifiable and privacy preserving federated learning without fully trusted centers," *J. Ambient Intell. Humanized Comput.*, vol. 13, pp. 1431–1441, 2022.

[31] A. Fu, X. Zhang, N. Xiong, Y. Gao, H. Wang, and J. Zhang, "VFL: A verifiable federated learning with privacy-preserving for big data in industrial IoT," *IEEE Trans. Ind. Informat.*, vol. 18, no. 5, pp. 3316–3326, May 2022.

[32] C. Hahn, H. Kim, M. Kim, and J. Hur, "VerSA: Verifiable secure aggregation for cross-device federated learning," *IEEE Trans. Dependable Secure Comput.*, vol. 20, no. 1, pp. 36–52, Jan./Feb. 2023.

[33] J. Zhang, Y. Liu, D. Wu, S. Lou, B. Chen, and S. Yu, "VPFL: A verifiable privacy-preserving federated learning scheme for edge computing systems," *Digit. Commun. Netw.*, vol. 9, pp. 981–989, 2023.

[34] Y. Wang, A. Zhang, S. Wu, and S. Yu, "VOSA: Verifiable and oblivious secure aggregation for privacy-preserving federated learning," *IEEE Trans. Dependable Secure Comput.*, vol. 20, no. 5, pp. 3601–3616, Sep./Oct. 2023.

[35] R. Wang, L. Xiong, J. Geng, C. Xie, and R. Li, "An effective and verifiable secure aggregation scheme with privacy-preserving for federated learning," *J. Syst. Archit.*, vol. 161, 2025, Art. no. 103364.

[36] H. Gao, N. He, and T. Gao, "SVeriFL: Successive verifiable federated learning with privacy-preserving," *Inf. Sci.*, vol. 622, pp. 98–114, 2023.

[37] Y. Zhang and H. Yu, "Towards verifiable federated learning," 2022, *arXiv:2202.08310*.

[38] F. Luo, H. Wang, and X. Yan, "Comments on "VERSA: Verifiable secure aggregation for cross-device federated learning"," *IEEE Trans. Dependable Secure Comput.*, vol. 21, no. 1, pp. 499–500, Jan./Feb. 2024.

[39] J. Wu and W. Zhang, "On the security of verifiable and oblivious secure aggregation for privacy-preserving federated learning," *IEEE Trans. Dependable Secure Comput.*, vol. 21, no. 5, pp. 4324–4326, Sep./Oct. 2024.

[40] R. Fotohi, F. Aliee, and B. Farahani, "Decentralized and robust privacy-preserving model using blockchain-enabled federated deep learning in intelligent enterprises," *Appl. Soft Comput.*, vol. 161, 2024, Art. no. 111764.

[41] Y. Qu, M. Uddin, C. Gan, Y. Xiang, L. Gao, and J. Yearwood, "Blockchain-enabled federated learning: A survey," *ACM Comput. Surv.*, vol. 55, 2022, Art. no. 70.

[42] J. Zhu, J. Cao, D. Saxena, S. Jiang, and H. Ferradi, "Blockchain-empowered federated learning: Challenges, solutions, and future directions," *ACM Comput. Surv.*, vol. 55, pp. 1–31, 2023.

[43] Y. Xu, C. Peng, W. Tan, Y. Tian, M. Ma, and K. Niu, "Non-interactive verifiable privacy-preserving federated learning," *Future Gener. Comput. Syst.*, vol. 128, pp. 365–380, 2022.

[44] C. Brunetta, G. Tsaloli, B. Liang, G. Banegas, and A. Mitrokotsa, "Non-interactive, secure verifiable aggregation for decentralized, privacy-preserving learning," in *Proc. Australas. Conf. Inf. Secur. Privacy*, 2021, pp. 510–528.

Authorized licensed use limited to: Anhui University. Downloaded on October 13,2026 at 08:43:08 UTC from IEEE Xplore. Restrictions apply.

[45] X. Yin, Y. Zhu, and J. Hu, "A comprehensive survey of privacy-preserving federated learning: A taxonomy, review, and future directions," *ACM Comput. Surv.*, vol. 54, pp. 1–36, 2021.
[46] A. Yao, "Protocols for secure computations," in *Proc. 23rd Annu. Symp. Found. Comput. Sci.*, 1982, pp. 160–164.
[47] S. Nakamoto, "Bitcoin: A peer-to-peer electronic cash system," 2008.
[48] H. Kalodner, M. Carlsten, P. Ellenbogen, J. Bonneau, and A. Narayanan, "An empirical study of namecoin and lessons for decentralized namespace design," in *Proc. 14th Annu. Workshop Econ. Inf. Secur.*, 2015, pp. 1–23.
[49] H. Fabric, "A blockchain platfrom for the enterprise," *Hyperledger*, 2023. Accessed: Jul. 18, 2025. [Online]. Available: https://hyperledger-fabric.readthedocs.io/en/release-2.5
[50] Y. Li, Y. Yu, C. Lou, N. Guizani, and L. Wang, "Decentralized public key infrastructures atop blockchain," *IEEE Netw.*, vol. 34, no. 6, pp. 133–139, Nov./Dec. 2020.
[51] R. Gennaro, S. Jarecki, H. Krawczyk, and T. Rabin, "Secure distributed key generation for discrete-log based cryptosystems," in *Proc. Int. Conf. Theory Appl. Cryptographic Techn.*, 1999, pp. 295–310.
[52] S. Caldas et al., "LEAF: A benchmark for federated settings," 2018, *arXiv: 1812.01097*.
[53] Y. LeCun, C. Cortes, and C. Burges, "MNIST handwritten digit database," 1998. [Online]. Available: http://yann.lecun.com/exdb/mnist
[54] A. Krizhevsky et al., "Learning multiple layers of features from tiny images," Toronto, ON, Canada, 2009.
[55] C. Dwork, "Differential privacy," in *Proc. Int. Colloq. Automata Lang. Program.*, 2006, pp. 1–12.
[56] R. Guerraoui, N. Gupta, and R. Pinot, "Byzantine machine learning: A primer," *ACM Comput. Surv.*, vol. 56, 7, pp. 1–39, 2024.
[57] A. Boussetta, El-E. Mhamdi, R. Guerraoui, A. Maurer, and S. Rouault, "AKSEL: Fast byzantine SGD," in *Proc. 24th Int. Conf. Princ. Distrib. Syst.*, 2021, Art. no. 8.
[58] K. Boubouh, A. Boussetta, Y. Benkaouz, and R. Guerraoui, "Robust P2P personalized learning," in *Proc. 2020 Int. Symp. Reliable Distrib. Syst.*, 2020, pp. 299–308.



Jaouhara Bouamama received the bachelor's degree in applied mathematics and the master's degree in cryptography and information security from the Faculty of Sciences, Mohammed V University in Rabat, Morocco, in 2016 and 2018. She is currently working toward the PhD degree with the Computer Science & Smart Systems Laboratory, Higher School of Technology, Hassan II University of Casablanca, Morocco. She was a research engineer with Mohammed VI Polytechnic University in Benguerir, Morocco, where she worked on cloud computing security. Her main research was on Trusted Execution Environments (TEEs) for secure key management in cloud infrastructures. Her current research interests lie in the intersection of applied cryptography, information security, and federated learning.



Yahya Benkaouz received the PhD degree from the ENSIAS, Mohammed V University in Rabat, Morocco. He is currently an associate professor with the Mohammed V University in Rabat and an affiliate professor with Mohammed VI Polytechnic University in Benguerir, Morocco. His PhD considered privacy-preserving protocols for distributed data aggregation and was defended in 2015. He was a visiting researcher with Several Research Labs and Groups at the EPFL in Switzerland, TU Darmstadt and Marburg University in Germany, and INRIA Rennes in France. He has been involved in the organization of several distributed computing conference venues: The international conference NETYS (since 2013) and the METIS School in Distributed Systems (Since 2010). He also co-founded the workshop BiDAS on Big Data Analytics, Security and Privacy. He served as a reviewer for multiple prestigious journals such as the *ACM Transactions on Privacy and Security*, *ACM Transactions on Information and System Security*, and *IEEE Transactions on Dependable and Secure Computing*. He is also a PC member for several CS conferences. His research interests include distributed systems, blockchain technology, security, and privacy.



Mohammed Ouzzif received the postgraduate doctorate degree from the Faculty of Sciences in Rabat, Morocco, in 1999, and the PhD degree from the ENSIAS, Mohammed V University in Rabat, in 2005. He is currently a professor with the Higher School of Technology, which he joined in 1996. His primary contributions are in the field of dynamic resource management within collaborative systems, with a particular focus on distributed mutual exclusion algorithms. His research interests include distributed systems and networks, and he has published extensively in these areas. He is a member of the Steering Committee of the SYSCO-BIOTS conference and has served on the Technical Program Committees of various international conferences. In 2021, he founded and currently heads the Computer Science & Smart Systems Laboratory and is a professional member of the IEEE Computer Society.

Authorized licensed use limited to: Anhui University. Downloaded on October 13,2026 at 08:43:08 UTC from IEEE Xplore. Restrictions apply.